Module 3 Consolidated Notes

Core Concepts of Large Language Models

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain what a large language model is and how next-token prediction drives everything it does
- Trace the full pipeline from raw text to a model's output — covering tokenization, embeddings, positional encoding, and the transformer architecture
- Apply prompt engineering techniques (zero-shot, one-shot, few-shot) and describe how Retrieval-Augmented Generation extends a model's knowledge
- Identify the key failure modes of LLMs — hallucination, bias, and privacy risks — and describe practical ways to reduce them

---

2. Detailed Explanation

What an LLM is — next-token prediction

A **large language model (LLM)** is a system that assigns a probability to every possible next word (or token) given everything that came before it. That single mechanism — predicting the next token — is what drives every capability you see: answering questions, writing code, summarising documents, translating languages.

Think of the auto-complete keyboard on a smartphone. After you type "Happy", it suggests "Birthday", "Anniversary", or "Wedding". An LLM does exactly this, but powered by trillions of learned associations rather than a small lookup table.

LLMs do not memorise sentences. They learn **useful language patterns** from massive amounts of text — much like a child who picks up a language by listening to it for years, recognising patterns without ever memorising an explicit grammar rule. The zebra analogy captures this well: a child shown a zebra and told "this is a zebra, not a horse" learns to recognise zebras by the pattern of black and white stripes. The next time they see a different zebra, they identify it by pattern — not by formula. LLMs work the same way.

Well-known LLMs include ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google), Perplexity, and Sarvam AI.

How LLMs differ from traditional NLP

Earlier **natural language processing (NLP)** systems were task-specific: each model was built for one narrow job (e.g., spam detection or machine translation), trained on a small dedicated dataset, and unable to generalise beyond that task.

An LLM is trained on a **massive, general-purpose dataset** — books, web pages, code, articles, and public documents — and learns a general representation of language. This means one model handles translation, summarisation, question answering, code generation, essay writing, and more simultaneously. This "one model, many tasks" property is the defining characteristic of the LLM era and a core shift from earlier NLP approaches.

Parameters, weights, biases, and activation functions

Parameters are the learnable numbers inside a model that get adjusted during training. In a simple equation like `y = mx + c`, `m` (the slope) and `c` (the intercept) are two parameters. Neural networks extend this idea: every connection between neurons carries a **weight**, and every neuron has a **bias** term. Weights and biases are both parameters.

The scale of parameters is what makes an LLM "large":

Model	Approximate parameter count
Simple whiteboard neural network	~17 parameters
Sarvam AI (Indian LLM)	~2 billion parameters
ChatGPT	~1 trillion parameters

At 1 trillion parameters, the model encodes an enormous graph of learned associations. This is also why running such a model demands large server infrastructure — it cannot run on a personal computer.

Neural networks also require **activation functions** — mathematical operations like ReLU, sigmoid, and tanh — to introduce non-linearity. Without them, stacking multiple layers would not increase the model's expressive power; everything would collapse into a single linear transformation.

Hyperparameters are different from parameters. A **hyperparameter** is a setting you choose *before* training begins — it is not learned from the data. The learning rate used in gradient descent is the classic example. Parameters (weights and biases) are the *outputs* of training; hyperparameters are the *controls* you set before training starts.

Tokenization — turning text into numbers

Before a model can do any mathematics, text must become numbers. **Tokenization** is the process of splitting text into small units called **tokens** and assigning each token a unique integer ID.

A token can be a complete word, a part of a word (a subword unit), a punctuation mark, a number, or a symbol. For example, the sentence "Welcome to the class" splits into four tokens: `welcome`, `to`, `the`, `class` — each with its own integer ID.

The full set of tokens a model recognises is called its **vocabulary**. If a vocabulary has 50,000 entries, the model chooses among 50,000 candidates at every prediction step. Tokenization is the essential bridge from human-readable text to the numerical domain that hardware can process.

Watch out for: Tokenization is not the same as splitting on spaces. Subword tokenization means that uncommon words (like names or technical terms) may be split into multiple tokens, which can affect how the model handles them.

Embeddings — from integers to vectors

After tokenization, each token is represented as a single integer (a **scalar**). A scalar carries no information about how words relate to each other semantically. **Embedding** converts each scalar token ID into a dense **vector** — a list of floating-point numbers.

For example, a token with ID 50 representing `king` might map to a vector like `[0.25, 0.25, ...]` in a high-dimensional space. Every token in the vocabulary gets its own embedding vector.

The key concept is that embedding places tokens in a **continuous mathematical space** where geometric proximity corresponds to semantic similarity. Words with related meanings end up closer together; words with different meanings end up farther apart. This is the foundation for everything the model can do with meaning.

Formally: embedding converts a scalar (token ID) into a vector — the token's representation inside the model's internal mathematical space.

Positional encoding — adding word order

Embeddings alone do not tell the model where in a sentence each word appears. Consider "dog bites man" versus "man bites dog" — same words, very different meanings. Throwing tokens into a bag without order destroys meaning.

To fix this, a **positional encoding** — a separate vector representing the token's position in the sequence — is added to each token's embedding. The input to the transformer therefore combines both pieces of information:

```
input_vector = token_embedding + positional_encoding
```

This combined vector tells the model both *what* the token is and *where* it sits in the sentence.

The transformer architecture

```
flowchart TD
    InputText["Input text"] --> Tokenization["Tokenization"]
    Tokenization --> TokenIDs["Token IDs (integers)"]
    TokenIDs --> EmbedEncode["Embedding + positional encoding"]
    EmbedEncode --> Attention["Multi-head self-attention"]
    Attention --> FeedForward["Normalisation + feed-forward layers"]
    FeedForward --> Logits["Output logits"]
    Logits --> Softmax["Softmax → probability distribution"]
    Softmax --> NextToken["Next token selected"]
```

Before transformers, sequence models used **recurrent neural networks (RNNs)** and **long short-term memory (LSTM)** networks. These processed tokens one at a time — sequentially — which made training slow and made it hard for the model to relate tokens that were far apart in a sentence.

The transformer was introduced in the 2017 paper **"Attention Is All You Need"**, published at NeurIPS and authored at Google Brain. Its core innovation was replacing recurrence with **self-attention**, which allows all tokens in a sequence to be processed in parallel. This made the architecture far faster to train and far better at capturing long-range dependencies.

Today every major LLM — including ChatGPT — uses the transformer as its backbone. GPT stands for **Generative Pre-trained Transformer**.

Issue with RNN/LSTM	Transformer's solution
Sequential processing — slow to train	Processes all tokens in parallel
Hard to relate distant tokens	Self-attention attends to all tokens regardless of distance
Difficult to scale	Architecture scales efficiently with data and compute

---

Self-attention — how tokens relate to each other

Self-attention is the core mechanism that lets every token gather context from every other token in the sequence. For each token, the model computes three vectors by multiplying the token's input vector by three separate learned weight matrices:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What information do I contain?"
- **Value (V):** "What information should I pass on?"

The attention mechanism compares the query of one token against the keys of all other tokens to produce **attention weights** (written α_ij for the weight from token i to token j). These weights determine how much each token's value flows into the contextualised representation of the querying token.

Consider the sentence "The animal did not cross the road because it was tired." The word `it` must be understood to refer to `animal`. Self-attention gives the model a mechanism to learn that `it` should attend strongly to `animal`. The weight α connecting `it` to `animal` becomes high, letting the meaning of `animal` flow into the representation of `it`.

Softmax in attention: The raw attention scores are real numbers ranging from −∞ to +∞. The **softmax function** converts them into a valid probability distribution — values between 0 and 1 that sum to 1:

```
softmax(vᵢ) = e^vᵢ / (e^v₁ + e^v₂ + e^v₃)
```

Softmax also appears at the very end of the transformer pipeline, where it converts the model's raw output scores into a probability distribution over the full vocabulary so the model can select the next token.

One important distinction: attention weights are computed fresh from the input on every forward pass. They are not stored as fixed parameters like weights and biases — they are dynamic, per-input computations.

Multi-head attention — seeing multiple relationships at once

Instead of computing a single set of Q, K, V vectors, the transformer computes **multiple sets in parallel** — these parallel sets are the "heads" in **multi-head attention**. Each head learns to attend to a different kind of relationship (for example, one head may focus on syntactic structure, another on coreference links like `it` → `animal`). The outputs of all heads are concatenated and projected to produce the final contextualised representation.

Multi-head attention gives the transformer three key advantages:

- Parallel processing of the full token sequence
- The ability to capture multiple types of long-range dependency simultaneously
- Better scalability compared to sequential architectures

Text generation — autoregressive prediction

An LLM generates text one token at a time in a loop called **autoregressive generation**:

1. The model receives the current sequence of input tokens.
2. It computes a probability distribution over the entire vocabulary for the next token.
3. The highest-probability token (or a sampled token) is selected and appended.
4. The extended sequence is fed back as the new input.
5. Steps 2–4 repeat until the output is complete.

For example, given "The capital of India is", the model assigns a probability of approximately 0.9 to `Delhi` and very low probabilities to alternatives like `Chandigarh`. It selects `Delhi` and continues.

Training penalises wrong predictions — and the penalty is harsher when the model was very confident but still wrong than when it was uncertain. This encourages the model to be calibrated: only assign high probability to tokens it genuinely believes are correct.

Pre-training and fine-tuning

Training an LLM happens in two main stages.

Pre-training is the first and most computationally intensive phase. The model trains on an enormous corpus — books, web pages, code, articles — by predicting the next token at every position. This produces a general-purpose model with broad language understanding and world

knowledge. ChatGPT's pre-training corpus spans domains ranging from C and Java source code to HTML, essays, and natural-language content across many subjects. Pre-training requires large server clusters and can take months.

Fine-tuning comes after. The pre-trained model is further trained on a smaller, curated dataset to specialise its behaviour — for example, to follow instructions, maintain a conversational tone, or focus on a domain like medicine or finance.

Temperature is a hyperparameter that controls how deterministically the model samples from its output distribution. At low temperature, the model consistently picks the highest-probability token (predictable, conservative output). At high temperature, it samples more randomly (creative but less reliable output). This is an instance of the exploration–exploitation trade-off familiar from reinforcement learning.

Prompt engineering — getting better outputs

A **prompt** is the text input you send to an LLM. The quality of the prompt directly shapes the quality of the output. An effective prompt may include: a role, a task description, the intended audience, relevant context, the desired output format, length constraints, and examples.

The three core prompting strategies are:

Strategy	What it means	Example
Zero-shot	No examples; the model relies entirely on its training	"Classify this review as positive or negative: 'The product is easy to use.'"
One-shot	One labelled example before the actual query	Show one classified review, then ask the model to classify a new one
Few-shot	Several examples to establish the pattern	"Excellent camera → positive; Battery drains fast → negative; [new review] → ?"

The **context window** is the maximum number of tokens the model can process in a single input–output exchange. Once the context window is full (for example, at 10,000 tokens), the model can no longer see earlier parts of the conversation. Very long prompts or conversations will push earlier content out of the window.

Retrieval-Augmented Generation (RAG)

RAG — Retrieval-Augmented Generation — combines an LLM with an external knowledge base. When a user submits a query, a retrieval system first searches the knowledge base (a research paper database, a business document repository, etc.) for relevant passages. Those passages are injected into the prompt, giving the LLM grounded, up-to-date, or proprietary context that was never part of its training data.

RAG is most useful when:

- The required information is recent and post-dates the model's training cutoff
- Citations to specific documents are needed
- Reducing hallucination is a priority

RAG is an add-on concept applied to the base LLM. Both components work together: the retrieval system finds relevant facts, and the LLM synthesises them into a coherent response.

Hallucination — confident but wrong

Hallucination is the phenomenon where an LLM produces a confident-sounding answer that is factually wrong. The mechanism is straightforward: the model assigns high probability to a token sequence that is linguistically fluent but factually incorrect.

A simple example: a model might state "The capital of France is New Delhi" — a sentence that is grammatically well-formed and follows a plausible pattern, but is wrong. In practice, hallucinations appear as:

- Fabricated research papers (plausible-sounding titles and author lists for papers that do not exist)
- Wrong dates, incorrect formulas, invented facts — all delivered with high apparent confidence

To reduce hallucination in practice:

1. Use RAG to ground responses in real documents
2. Verify critical facts independently
3. Use hedging language when uncertain ("I think", "I'm not fully certain")
4. Write specific prompts — vague prompts give the model more room to speculate

---

Bias and privacy risks

Bias in LLMs means the model's outputs can systematically favour one viewpoint. This happens because LLMs reflect biases present in their training data or introduced through post-training alignment choices. Examples include geopolitical sensitivity (skewed or evasive answers about political topics), cultural bias, and gender bias. DeepMind, for instance, has tuned some generative models to avoid answering questions about certain political topics — a form of intentional bias introduced through post-training constraints.

Bias is distinct from hallucination: a biased answer may be factually defensible but still systematically favours one perspective.

Privacy risk is a separate concern. LLMs retain context within a conversation and, in some configurations, across sessions. Sharing sensitive personal or professional information — such as confidential business strategies, personal identification details, or proprietary research — with a public generative AI model creates a real risk. That information may be stored, used in subsequent responses, or exposed to others.

Practical rule: do not share critical personal or confidential information with any public generative AI model.

Limitations of LLMs

LLMs are powerful tools, but they have well-documented limitations:

Limitation	Explanation
Hallucination	Confident but factually wrong answers
Limited real-time knowledge	Training data has a cutoff date; the model does not know recent events
Bias	Training data and alignment choices introduce systematic skews
Lack of true reasoning	Pattern-matching is not the same as logical deduction
Privacy risk	Sensitive information shared in prompts may be retained
Not suitable for high-stakes autonomous decisions	Domains like medical diagnosis and defence require verified expertise and accountability

LLMs are tools to augment learning and productivity — not authoritative oracles.

Small language models and the future of AI

General-purpose LLMs like ChatGPT (~1 trillion parameters) are designed to handle any topic. But smaller, **domain-specific models** trained on targeted corpora can outperform large general models within their specialty. Examples include medical, financial, educational, and defence language models.

The concept of **dedicated small language models** is compelling: they require far less compute to train and deploy, produce more accurate results within their domain, and are easier to audit for accuracy and bias.

Perplexity — measuring how well a model predicts

Perplexity is a technical metric that measures how well a language model predicts a sequence of tokens. Lower perplexity means the model is better at predicting the text — it is less "surprised" by what comes next. Perplexity is derived from the cross-entropy of the model's predicted probability distribution against the actual token sequence. The Perplexity search engine is named after this concept and uses it as part of its model evaluation framework. Understanding the concept of perplexity helps interpret how confident a model is in its predictions.

Common misconceptions

Misconception	Correction
"LLMs are internet search engines"	LLMs generate text from learned parameters; they do not query the live internet unless RAG is explicitly integrated
"LLMs always give the correct answer"	LLMs predict the most probable next token, which can be wrong; hallucination is a well-documented failure mode
"Bigger LLMs are always better"	Domain-specific small models frequently outperform large general models within their specialty

---

3. Key Takeaways

- An LLM's entire capability — answering, writing, translating, coding — emerges from one mechanism: predicting the next token using learned probability distributions over a vocabulary.
- The pipeline from text to output flows through tokenization, embedding, positional encoding, transformer layers (with self-attention and feed-forward), and softmax.

- Understanding each stage in the pipeline is key to grasping how LLMs work.
- Self-attention and multi-head attention are the transformer's core power: every token attends to every other token in parallel, capturing both local and long-range relationships simultaneously.
- Prompt engineering (zero-shot, one-shot, few-shot) and RAG are the main tools for getting reliable, grounded outputs from an LLM in practice.
- LLMs have real failure modes — hallucination, bias, knowledge cutoffs, and privacy risks — and knowing them helps you use these tools responsibly rather than blindly.

Mental model: Think of an LLM as a colossal pattern library built from trillions of text examples. Every response it generates is the system finding the most statistically plausible continuation of your input.

Prompt Engineering and Reasoning Strategies

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Build well-structured prompts using the five-component skeleton (Role, Context, Task, Format, Length) for real API use
- Apply zero-shot, one-shot, and few-shot techniques to control model output format and reasoning
- Use chain of thought and task decomposition to tackle complex, multi-step problems
- Call the OpenAI and Groq APIs in Python, handle prompt roles, and manage token costs

---

2. Detailed Explanation

What prompt engineering is — and why it matters at the API level

Prompt engineering is the practice of crafting instructions to a large language model (LLM) so it produces consistent, high-quality, and controlled outputs. Think of it as the difference between shouting a vague request across a room versus handing a colleague a clear, written brief.

When you use a product like ChatGPT through its web UI, much of the formatting and safety work happens automatically behind the scenes. When you call an LLM via API — to build your own application — those automatic adjustments are absent. Every prompt detail must be specified explicitly. That is why structured prompt engineering matters most in API-driven development.

Before writing a single prompt, it helps to understand what an LLM actually does. A model like ChatGPT, Gemini, Claude, or one accessed via Groq **predicts output one token at a time**. Given the input "What is the capital of India?", the model does not look up a fact in a database. It predicts the most probable next token (a letter or word fragment), then the next, and continues until it reaches a natural stopping point. The answer "New Delhi" is predicted — not retrieved — because that sequence was statistically likely in the training data.

This matters for prompt engineering: the way you phrase an instruction steers which probability patterns the model follows.

The three prompt roles: system, user, and assistant

When you call an LLM API, you pass a list of messages. Each message has a **role** field. There are exactly three roles, and each does a different job:

```
flowchart LR
    SystemRole["System role\n(sets persona & scope)"] --> Model["LLM processes\nall three t
    UserRole["User role\n(question or task)"] --> Model
    AssistantRole["Assistant role\n(prior responses & history)"] --> Model
    Model --> Output["Generated response"]
```

System role sets the persona of the LLM for the entire conversation. LLMs are trained on data spanning medicine, engineering, law, design, and more — so by default they answer from a general-purpose perspective. The system prompt narrows that scope. If you want the model to respond as a back-end cloud engineer, a doctor, or a patient teacher for beginners, put that instruction here.

User role carries the actual question or task — the input your end-user or application sends.

Assistant role holds previous model responses. An LLM has no persistent memory of its own. When you build a multi-turn chat interface, your application re-sends the entire conversation history — prior user messages and prior assistant responses — with every new request. The model reads that history as assistant-role messages. It can then answer questions like "What was my first question?" by scanning the conversation it was handed.

Watch out for: Forgetting to include prior conversation turns when building chat applications. If you only send the latest user message, the model has no history to refer back to, and multi-turn conversations will feel disjointed.

The prompt skeleton: Role, Context, Task, Format, Length

Every non-trivial prompt you send via API should contain five components. Here is what each one does:

Component	What it means
Role	The persona you want the LLM to adopt — e.g., "You are an AI engineer with five years of production experience."
Context	Background information the model needs. Think of it as an employee onboarding knowledge transfer — you cannot expect good output without relevant facts.
Task	The specific thing you want the model to do — e.g., "Write a cold email to the HR recruiter for this role."
Format	How you want the output structured — a JSON object, a markdown table, a numbered list, a subject/body/sign-off email.
Length	How long the output should be. This is tied directly to token cost — longer outputs cost more.

To see the difference in practice, compare these two prompts:

Bad prompt: "Write a cold email for an engineering job."

This has no role, no context, no format guidance, and no length constraint. The output will be generic and impersonal.

Good prompt (built around the skeleton): - *Role:* "You are an AI engineer with five years of production experience writing a cold email to a recruiter. Be confident, specific, and not desperate." - *Context:* The target audience (HR recruiter), the writer's background, the goal, and the job description pasted in. - *Task:* "Write a cold email to the HR recruiter for this role." - *Format:* Subject line → personalised greeting → short body (one major project, one clear call to action) → sign-off. - *Length:* Under five sentences for the body.

The well-structured version produces output that includes the recruiter's company name, the writer's relevant experience, a specific project with results, and a request for a 15-minute screening call — a measurable improvement over the generic version.

The same principle applies to technical tasks. A bad prompt — "How do I handle errors in Python?" — gives no persona, no code context, no format. Adding a system message that establishes a senior Python developer persona and pasting in the relevant code block causes the model to provide deeper, production-quality guidance rather than a beginner overview.

Constraints and negatives

A strong prompt specifies not only what the output should contain but also **what it should not contain**. Examples of constraints:

- No emojis in the output.
- No hyphens.
- No rude or overly formal tone.
- Output only "positive", "negative", or "neutral" — no extra explanation.

Constraints are especially important when the model's output feeds directly into downstream code. If your application parses the response programmatically, an unexpected extra sentence or emoji will break the parser. Specifying negatives upfront prevents that.

Persona vocabulary — keywords that reliably steer model behaviour

When you write the system prompt, certain role-type keywords produce predictably different behaviours:

Keyword	Effect on model behaviour
Senior developer / expert	Technical depth, production-quality explanations
Teacher	Simple analogies, beginner-friendly language
Critic	Validation and error-finding mindset
Translator	Language conversion tasks
Simplifier	Reduced complexity, plain language
Brainstormer	Multiple ideas or alternatives
Debater	Argues multiple sides of a question

Choosing the right keyword is a small change that produces a large difference in tone and depth.

Chain of thought and task decomposition

Chain of thought (CoT) is a technique where you instruct the model to reason step by step before producing its final answer. It is especially valuable for complex, multi-stage tasks where a direct-answer approach skips crucial intermediate reasoning.

To see why decomposition matters, consider a content creator who wants to produce a YouTube video on embeddings. The full pipeline involves five distinct steps: 1. Reading a prior video to learn the creator's style 2. Writing the script 3. Validating the code and script 4. Generating the YouTube description 5. Designing the thumbnail concept

Bundling all five tasks into a single prompt is like hiring one person to simultaneously act as content creator, scriptwriter, thumbnail designer, and code validator. The output will be shallow across all five dimensions.

The CoT or **task decomposition** solution breaks the work into individual prompts. The output of prompt 1 becomes the input context for prompt 2, and so on. Each step gets the model's full attention and produces controlled output in a format the next step can consume.

In code, adding the phrase "solve by thinking step by step" causes the model to emit its full reasoning chain before the final answer:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "user",
            "content": "Solve by thinking step by step: extract product name, price, and se
        }
    ]
)
```

This approach improves extraction accuracy on structured tasks by guiding the model through the inference rather than letting it jump to a conclusion.

Reasoning models and CoT: Newer reasoning-class models — GPT-5.5, Claude 4.8 (Opus-level), and similar — apply chain-of-thought reasoning by default without an explicit "step by step" instruction. When you work with older or smaller models via the API, the explicit CoT prompt is still necessary.

Zero-shot, one-shot, and few-shot prompting

These terms describe **how many examples** you include in the prompt to show the model the desired output format or reasoning pattern.

Term	Meaning
Zero-shot	No examples provided; the model must infer the expected output from the instruction alone.
One-shot	One example of question → answer is provided.
Two-shot	Two examples provided.
Few-shot	A small number of examples (typically 2–5).

Format control with few-shot — a worked example. Without examples, asking "What are the famous places in Hyderabad?" returns a simple bullet list. Providing one example answer — three points, each with a one-line description (e.g., "Charminar — best place for shopping") — and then asking about Bangalore causes the model to match that exact structure. No explicit format instruction is needed.

Structured JSON extraction with few-shot via the messages list. Few-shot examples in API calls are passed as actual messages. You add prior user and assistant turns that demonstrate the pattern, then append the real question:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "system",
            "content": "Extract product info as JSON with keys: product, price, sentiment."
        },
        {"role": "user", "content": "Got a card for 15,999 and it's amazing"},
        {
            "role": "assistant",
            "content": '{"product": "Samsung M4", "price": 15999, "sentiment": "positive"}'
        },
        {"role": "user", "content": "Camera is okay but battery is the worst"},
        {
            "role": "assistant",
            "content": '{"product": "OnePlus Nord", "price": 22999, "sentiment": "negative"
        },
        {"role": "user", "content": "Camera is great and battery is amazing"}
    ]
)
```

The two assistant-role examples teach the model the exact JSON schema, so the response to the real question follows the same structure reliably — something zero-shot rarely achieves without an explicit format instruction.

Calling LLM APIs in Python

Here is the practical scaffolding for the two providers covered — OpenAI and Groq.

Install the packages:

```
pip3 install openai
pip3 install groq
```

Initialise the OpenAI client and make a basic call:

```
from openai import OpenAI

client = OpenAI(api_key="<your-openai-api-key>")

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "What is machine learning in one sentence?"}
    ]
)
print(response.choices[0].message.content)
```

The model returns a `ChatCompletion` object. The generated text is at `response.choices[0].message.content`.

Initialise the Groq client with a system persona:

```
from groq import Groq

client_groq = Groq(api_key="<your-groq-api-key>")

response = client_groq.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "system",
            "content": "You are a patient teacher for beginners. Use simple analogies and k
        },
        {"role": "user", "content": "How do I handle missing data in machine learning?"}
    ]
)
```

Groq uses the same chat completions interface as OpenAI — the message structure is identical; only the client initialisation and model name differ.

Sentiment classification loop — zero-shot with a constraint:

```
reviews = [
    "Terrible service, never coming back",
    "It was okay, nothing special",
    "Best biryani in Hyderabad",
    "The delivery was late but the food was good"
]

for review in reviews:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "user",
            "content": f"Classify as positive, negative, or neutral. No extra text.\n\n{rev
        }]
    )
    print(review, "→", response.choices[0].message.content)
```

The constraint "No extra text" is the key — without it, the model may explain its reasoning and break any downstream parser that expects a single word.

Groq rate limits (Llama 3.3 70B Versatile, free tier)

When using `llama-3.3-70b-versatile` on Groq's free tier, you are subject to these limits:

- 30 requests per minute
- 1,000 requests per day
- 12,000 tokens per minute
- 100,000 tokens per day

These limits appear in the Groq console. Factor them into any application that calls the model in a loop — hitting the per-minute limit will return an error and break your processing pipeline.

Token cost and why well-engineered prompts save money

LLM API pricing has two components: **input tokens** (text you send) and **output tokens** (text the model generates). Output tokens cost significantly more. GPT-5.5 illustrates the asymmetry:

- **Input:** $5 per million tokens
- **Output:** $30 per million tokens — six times the input cost

Writing a longer, well-structured prompt (more input tokens) is the cheap option. Getting a longer, uncontrolled response (more output tokens) is the expensive one. Specifying length constraints in the prompt directly reduces cost. Iterating with short, vague prompts wastes output tokens on repeated useless responses; a well-structured prompt saves money overall.

Microsoft and Uber have reportedly stopped certain LLM subscription arrangements because of cost — illustrating that token economics matter at scale.

Using LLMs to write better prompts

If you have a task description but are unsure how to structure it as an API prompt, paste the description into ChatGPT and ask it to convert your description into a structured prompt. For example, this task description:

> "I have a markdown file and I want to convert it to HTML using the `weasyprint` package and a virtual environment. Please provide the complete solution. Now convert this into a prompt so I can use it via API."

...returned a fully structured prompt covering role (expert Python developer), context (task and environment requirements), task (produce the solution), and format (code + instructions). LLMs can themselves act as prompt-engineering tools.

LangChain and multi-provider consistency

LangChain is a framework that provides a single, unified code interface for interacting with multiple LLM providers — OpenAI, Groq, Gemini, and others. Without it, switching providers requires rewriting client initialisation and API call patterns. With LangChain, swapping a provider is a single configuration change while the rest of the code stays the same. LangChain also integrates with LangSmith (observability) and includes built-in prompt management utilities.

Even without LangChain, be aware that different providers use different method names:

Provider	Chat completions method
OpenAI / Groq	`client.chat.completions.create(...)`
Gemini	`client.models.generate_content(...)` or `client.chat.send_message(...)`

Always check the provider's official documentation before writing integration code. Method signatures, authentication patterns, model names, and rate limits change frequently as providers iterate on their APIs.

Evaluating and refining your prompts

A prompt is considered well-engineered when it produces **consistent results across multiple runs** — not just one good output. The evaluation process:

1. Write a prompt using the full skeleton plus constraints and few-shot examples as needed.
2. Run the prompt multiple times.
3. If the output is consistent and correct each time, the prompt is well-engineered.
4. If output varies significantly, refine the role, context, or constraints and repeat.

At scale, a systematic approach helps. Andrej Karpathy built a tool called **LLM Council** that benchmarks multiple LLMs against a shared set of questions, using an additional LLM as an evaluator. For most projects, testing across your own question-answer dataset and using a separate LLM as a judge is the recommended approach.

The six major techniques and when to reach for each:

Technique	When to use
Skeleton (Role + Context + Task + Format + Length)	Every non-trivial prompt via API
Constraints / negatives	When output format must be controlled or certain content excluded
Zero-shot / one-shot / few-shot	When the model needs to learn a specific output format or domain pattern
Chain of thought	Complex multi-step reasoning or structured extraction tasks
Task decomposition	When a task has 3+ distinct sub-tasks that would overwhelm a single prompt
Using LLMs to write prompts	When you have a task description but lack the prompt structure

These strategies are not mutually exclusive. A single production prompt might combine a skeleton structure with few-shot examples and a CoT instruction. Prompt engineering improves with practice — the measure of quality is consistent, accurate output across repeated runs.

Practical activity: Testing prompt strategies

Build a prompt testing harness that compares zero-shot, one-shot, few-shot, and chain-of-thought strategies on a customer support message classification task. Use the OpenAI API or Groq API. Run each strategy on the same set of customer messages and record the outputs. Determine which strategy produces the most consistent and accurate classifications, and share the results with reasoning to justify your choice. This exercise will show, in concrete terms, how prompt engineering choices directly impact model consistency.

---

3. Key Takeaways

- Prompt engineering is essential at the API level. Products like ChatGPT apply UI-level optimisations automatically; when you call the API directly, every detail must be specified in your prompt.
- The five-component skeleton (Role, Context, Task, Format, Length) plus constraints is the foundation of every well-engineered prompt. Persona keywords like "senior developer", "teacher", or "critic" reliably steer model behaviour.
- Few-shot prompting — passing example question/answer pairs as messages — is the most reliable way to lock in a specific output format without verbose format instructions.
- Chain of thought ("think step by step") and task decomposition both address complexity: CoT improves single-prompt reasoning; decomposition splits a multi-stage task across multiple prompts where each output feeds the next.
- Output tokens cost six times more than input tokens (GPT-5.5 pricing). A longer, well-structured prompt is always cheaper than repeated iterations with a poorly formed one.

Mental model: Think of prompt engineering as writing a job brief for a capable but context-blind contractor. The more precisely you specify the role, background, deliverable, format, and constraints, the less time you waste correcting generic output.

Fundamentals of AI Agents and Tool Usage

1. What You'll Learn in This Section

Understanding the fundamentals of AI agents is the first step toward building systems that go beyond simple question-and-answer. In this lesson, you'll learn to:

- Explain the difference between a chatbot and an AI agent, and why that difference matters in practice

- Apply the ReAct pattern (Reason + Act) to understand how agents think, act, and observe in a loop
- Define Python tool functions and register them using a JSON schema, enabling correct tool usage by the LLM
- Build an autonomous agent loop that runs tools repeatedly until a task is complete

---

2. Detailed Explanation

LLMs — The Brain Behind Every AI System

A **large language model (LLM)** is the reasoning engine at the core of any AI application. You send it a prompt — a question or instruction — and it returns an answer. Well-known LLMs include GPT (OpenAI), Gemini (Google), Groq, Llama (Meta), and Claude (Anthropic). Regardless of provider, the interaction model is the same: send a prompt, receive output.

There is one important distinction between using an LLM through a product interface (like ChatGPT) and using it through an API. In a product UI, many capabilities are pre-wired for you. Through the API, you get the raw language model only — any extra capability you want, such as searching the web or running a calculation, must be explicitly provided as a **tool**. A fundamental grasp of this boundary is essential for building agents from scratch.

Chatbot vs. Agent — What the Loop Changes

A **chatbot** follows a simple request-response cycle:

```
User question → LLM → Answer
```

An **agent** adds two things: tools the LLM can call, and a loop that keeps going until the task is done.

```
User question → LLM → decide which tool(s) to call
                    → execute tool(s) → observe result
                    → think again → … (repeat until done)
                    → final answer
```

The loop is the defining feature. A chatbot answers in one shot. An agent can call multiple tools in sequence, check intermediate results, and only return a final answer when it is confident the task is complete.

The ReAct Pattern — Reason + Act

ReAct (Reason + Act) is the operating pattern that makes an agent's loop coherent. Every step inside the loop has three phases:

Phase	What happens
Thought	The LLM reasons about what it needs to do next.
Action	The LLM decides to call a tool and specifies which arguments to pass.
Observation	The tool runs and returns a result; the LLM observes that result.

The LLM then loops back to the Thought phase until it decides the work is done and produces a final answer.

Here is a concrete example: a user asks for the average salary in a dataset. The LLM thinks "I need to compute an average over a column" and calls a statistics tool with the column name. It observes the numeric result, thinks "I now have the answer", and responds to the user.

Tools — What They Are and What They Can Do

A **tool** is a Python function you define. The LLM cannot execute code itself; it can only decide which function should be called and with which arguments. Your code — running in the execution environment — actually runs the function.

This means tools can do anything a Python function can do:

- Perform mathematical calculations
- Look up weather or other live data
- Query a database
- Create PDFs or presentations
- Make HTTP requests to external APIs
- Book tickets through a third-party service

The boundary is always the same: the LLM reasons, your code acts.

Environment Setup

The code examples below run in **Google Colab**. Install the OpenAI Python package, then import the required modules:

```
pip install openai
```

```
import json
import time
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
```

If you do not have an OpenAI key, **Groq** exposes an OpenAI-compatible API. The only differences are the `api_key` and a `base_url` pointing to Groq's endpoint:

```
groq_client = OpenAI(
    api_key=GROQ_API_KEY,
    base_url="https://api.groq.com/openai/v1"
)
```

OpenRouter is another alternative that provides free API credits and access to many models through the same OpenAI-compatible interface.

The Messages Format and Basic Chat Completion

Every API call sends a `messages` list — a list of dictionaries, each with two keys: `role` and `content`.

- `"system"` — sets the LLM's persona and high-level instructions.
- `"user"` — contains the actual question from the user.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant. Use tools when needed."},
    {"role": "user",   "content": "What is 15 * 47 * 23 + 89?"}
]
```

Without tools, you call the API like this:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages
)
choice = response.choices[0]
```

When no tools are provided, the `finish_reason` field on the response is `"stop"` and the `content` field holds the direct answer.

Defining Tool Functions

Two Python functions serve as tools in the first agent.

`calculate` evaluates a mathematical expression and returns the result as a string:

```
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression and return the result."""
    try:
        result = eval(expression)
        return str(result)
    except Exception as e:
        return str(e)
```

`get_weather` looks up weather for a city using fake demo data:

```
def get_weather(city: str) -> str:
    """Return weather for a city using fake data for demo purposes."""
    fake_weather = {
        "hyderabad": "32°C, partly cloudy, humidity 65%",
        "bangalore": "26°C, partly cloudy, humidity 50%",
        "mumbai":    "30°C, partly cloudy, humidity 85%",
        "peddapuram": "38°C, sunny, humidity 75%",
        "chennai":   "39°C, sunny, humidity 85%",
        "delhi":     "25°C, clear, humidity 45%",
    }
    city_lower = city.lower().strip()
    if city_lower in fake_weather:
        return fake_weather[city_lower]
    return f"Weather data not available for {city}"
```

Notice that both functions return strings. This is not optional — LLMs are text-based, and passing integers or other types can cause errors. **Always return strings from tool functions.**

The `get_weather` function normalizes the input with `.lower().strip()` before the lookup, which prevents mismatches from capitalization or accidental spaces.

Tool Schema — How the LLM Learns About Your Tools

The LLM understands your tools through a **tool schema**: a JSON structure that describes each function's name, purpose, and parameters. You pass this schema to the API alongside your messages.

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "calculate",
            "description": "Evaluate mathematical expressions and return the result. "
                           "Use this for math computations. Example: '25 * 4 + 100'.",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "The mathematical expression to evaluate."
                    }
                },
                "required": ["expression"]
            }
        }
    }
]
```

The `description` field is critical. It acts like natural-language prompting — it tells the LLM in plain English when and how to use the tool. A vague description leads to a confused agent; a precise description guides the LLM to correct tool usage at the right moment. In fundamental terms, the quality of your descriptions directly determines the quality of the agent's decisions.

Calling the API with Tools and Reading `finish_reason`

Pass `tools=tools` to `chat.completions.create` to enable tool use:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    tools=tools
)
choice = response.choices[0]
```

The `finish_reason` field now tells you what the LLM decided:

`finish_reason`	**Meaning**
`"stop"`	LLM answered directly; no tool calls needed.
`"tool_calls"`	LLM decided to call one or more tools; your code must execute them.

When `finish_reason == "tool_calls"`, the response contains the tool name, the arguments the LLM chose to pass (as a JSON string), and a `tool_call_id` that links the result back to this specific call.

Manual Tool Execution — The Full Round-Trip

Before wrapping everything in a loop, it helps to see one complete round-trip manually. The steps below execute a single tool call and get the final answer:

```
# 1. Map function names to callables
available_tools = {"calculate": calculate, "get_weather": get_weather}

# 2. Extract tool call details from the LLM response
tool_call     = choice.message.tool_calls[0]
function_name = tool_call.function.name
arguments     = json.loads(tool_call.function.arguments)

# 3. Execute the tool
result = available_tools[function_name](**arguments)

# 4. Append the assistant turn and the tool result to message history
messages.append(choice.message)
messages.append({
    "role":         "tool",
    "tool_call_id": tool_call.id,
    "content":      result
})

# 5. Call the LLM again with the full updated history
final_response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    tools=tools
)
print(final_response.choices[0].message.content)
```

Two "why" questions always come up here.

Why append the tool result to messages? The LLM has no memory between API calls. The entire conversation — original question, assistant's tool decision, tool result — must be passed as context on every single call. The tool result uses `role: "tool"` and carries the `tool_call_id` so the LLM can match the result to the tool it requested.

Why `role: "tool"` specifically? The API requires a matching `tool` message for every `tool_calls` response. Without it, the LLM does not know whether its tool request was executed.

The diagram below shows how the message history grows across one round-trip:

```
sequenceDiagram
    participant User
    participant Agent Code
    participant LLM
    participant Tool Function

    User->>Agent Code: "What is 15 * 47 * 23 + 89?"
    Agent Code->>LLM: messages=[system, user], tools=[calculate, get_weather]
    LLM-->>Agent Code: finish_reason="tool_calls", tool=calculate, args={expression: "15*47
    Agent Code->>Tool Function: calculate("15*47*23+89")
    Tool Function-->>Agent Code: "16304"
    Agent Code->>LLM: messages=[system, user, assistant_turn, tool_result]
    LLM-->>Agent Code: finish_reason="stop", content="The answer is 16304."
    Agent Code->>User: "The answer is 16304."
```

The Autonomous Agent Loop — `run_agent`

Wrapping the round-trip logic inside a `for` loop creates a fully autonomous agent that keeps running until either the LLM signals it is done or the maximum step count is reached.

```
def run_agent(user_message: str, max_steps: int = 10, verbose: bool = True):
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use tools when needed."
        {"role": "user",   "content": user_message}
    ]
    for step in range(max_steps):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=tools
        )
        choice = response.choices[0]

        if choice.finish_reason == "stop":
            print("Final answer:", choice.message.content)
            return

        if choice.finish_reason == "tool_calls":
            messages.append(choice.message)
            for tool_call in choice.message.tool_calls:
                function_name = tool_call.function.name
                arguments     = json.loads(tool_call.function.arguments)
                result = available_tools[function_name](**arguments)
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
    print("Agent reached maximum steps.")
```

The three parameters control behavior:

Parameter	Purpose
`user_message`	The question or task from the user.
`max_steps`	Maximum LLM-plus-tool iterations before stopping (default 10).
`verbose`	Reserved for controlling debug output; not yet wired to print statements in this snippet.

The loop itself follows five steps on every iteration: call the LLM, check `finish_reason`, if `"stop"` print and exit, if `"tool_calls"` execute every tool and append results, then repeat.

Example invocations show how the same agent handles different question types:

```
run_agent("What is 15 * 47 * 23 + 89?")   # calls calculate tool
run_agent("What is the weather in Hyderabad?")  # calls get_weather tool
run_agent("What is Python?")               # no tool needed, answers directly
```

When the question is pure general knowledge, the LLM returns `finish_reason == "stop"` immediately without calling any tool at all.

Data Analysis Agent — Same Pattern, New Domain

The ReAct loop is not specific to math or weather. The exact same architecture works for any domain — including data analysis. A second agent operates on an in-memory employee dataset with columns such as name, department, and salary.

Three tool functions are defined for this agent:

Tool	What it does
`get_stats(column)`	Returns minimum, maximum, sum, average, and count for a numeric column.
`count_rows(filter_column, filter_value)`	Returns the number of rows where a column matches a value.
`list_columns()`	Returns the list of column names in the dataset.

Each function is registered in the tool schema exactly as before — name, description, parameters, required fields. The tool usage pattern is identical regardless of domain: the LLM decides which tools to call and in what order, based on the user's question:

User question	Tools called
"What is the average salary?"	`list_columns`, then `get_stats("salary")`
"How many people are in Engineering?"	`count_rows("department", "Engineering")`
"Which department has higher salaries?"	`get_stats` and `count_rows` across departments

The developer's code executes the functions; the LLM only reasons about which ones to invoke and in what sequence.

MCP — Extending Tool Calling to Remote Servers

MCP (Model Context Protocol) extends the same tool-calling concept to a network. When a tool requires code execution on a remote machine — for example, when the LLM writes code and needs to run it somewhere — a server must execute that code. An MCP server exposes tools over a network protocol so an LLM can call them remotely, following the same fundamental tool usage contract as local functions.

The fundamental pattern is identical to local tool calling: the LLM decides, the server executes, and the result is returned. MCP is not a different paradigm — it is the same ReAct loop operating across a network boundary. Once you understand the fundamentals covered in this lesson, extending to MCP is straightforward.

---

3. Key Takeaways

- An **agent** differs from a chatbot by using a loop: it calls tools, observes results, reasons again, and only responds when the task is truly complete.
- The **ReAct pattern** (Thought → Action → Observation) gives the loop structure. Every iteration follows these three phases until `finish_reason == "stop"`.
- **Tool functions are plain Python functions** — the LLM decides which one to call, but your code does the actual execution. Always return strings so the LLM can read the result.
- The **tool schema's description field** is the most important part of tool registration. It guides the LLM on when and how to use each tool, acting like natural-language prompting.
- The **message history accumulates the full conversation**. Every LLM call must include the original question, all assistant tool decisions, and all tool results — the LLM has no memory between calls.
- The same ReAct agent architecture scales to any domain — math, weather, data analysis, API calls — simply by adding new tool functions and registering them in the schema. Tool usage patterns stay identical regardless of domain.

Mental model: Think of an AI agent as a thoughtful intern who cannot touch the keyboard — they tell you exactly which command to run and what arguments to use, you run it and read them the output, and they keep directing you until the job is done.

Structured outputs and tool integration

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why LLMs produce variable outputs and why that breaks multi-agent pipelines
- Apply JSON schema structured outputs to lock down the exact shape and types an LLM returns
- Build output validation logic that catches bad responses before they reach the next agent
- Design idempotent tools that can be called multiple times without causing accidental harm

2. Detailed Explanation

Why LLM outputs keep changing

Large language models (LLMs) are probabilistic models — they do not produce a fixed answer for a fixed question. Every time you send the same prompt, the model samples from a probability distribution, so you can get a slightly different response each run.

Ask an LLM "what is the average population of India and how does it compare to the world?" twice and you might get one answer with five bullet points and a second with a short paragraph. The intent is the same; the format and wording differ.

This is fine when a human reads the answer. It becomes a serious problem the moment another piece of code has to *parse* that answer.

```
flowchart TD
    Question["User question"] --> LLM1["LLM (Agent 1)"]
    LLM1 -->|"Variable free-text output"| Agent2["Agent 2"]
    Agent2 -->|"Tries to parse → may fail"| Broken["Pipeline breaks"]
```

Imagine a booking assistant where Agent 1 searches for trains and Agent 2 actually books the ticket. If Agent 1 sometimes returns `"₹ 750"` and other times `"750 rupees"`, Agent 2 cannot reliably extract the price. Even a 10 % failure rate means one in ten users hits a broken experience in production.

Temperature — controlling how random the output is

Temperature is a hyperparameter that controls the randomness and creativity of an LLM's response. The value runs from 0 to 2.

Temperature	Effect	Good for
0	Highly deterministic — same prompt → nearly same output	Structured extraction, factual Q&A
0.3–0.5	Mild creativity	General assistants
1.0	Default (GPT-4o mini default is 1.0)	Open-ended tasks
2.0	Very creative / unpredictable	Brainstorming, poetry

Setting temperature to 0 reduces randomness but does not eliminate it — LLMs are still probabilistic models at their core, so a 10 % variation chance can remain.

Newer **reasoning models** (such as the OpenAI o-series — o1, o3, o4-mini — and Claude's extended-thinking variants) replace the temperature knob with a `reasoning_effort` parameter set to `low`, `medium`, or `high`. The idea is the same: control how much deliberation the model applies.

Watch out for: Temperature reduces variation but cannot guarantee a fixed output format. For production pipelines you need structured outputs (described next) on top of a low temperature.

JSON mode vs. JSON schema — two levels of format control

There are two ways to ask an LLM to produce JSON, and they offer very different guarantees.

JSON mode (`response_format: { type: "json_object" }`) tells the model to respond with valid JSON. It is quick to set up and good for rapid prototyping, but it does not enforce which keys appear or what types the values have.

JSON schema (`response_format: { type: "json_schema", ... }`) is the production-grade option. You supply an explicit schema that names every field, its type, and an optional description. The model is constrained to match that shape exactly.

Here is a complete example for extracting product purchase information:

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "product_extraction",
        "strict": True,
        "schema": {
            "type": "object",
            "properties": {
                "product_name": {
                    "type": "string",
                    "description": "Name of the product"
                },
                "price_per_unit": {
                    "type": "number",
                    "description": "Price of one unit in INR"
                },
                "quantity": {
                    "type": "number",
                    "description": "Number of units requested"
                },
                "total_amount": {
                    "type": "number",
                    "description": "Total cost: price_per_unit multiplied by quantity"
                }
            },
            "required": ["product_name", "price_per_unit", "quantity", "total_amount"],
            "additionalProperties": False
        }
    }
}
```

Pass this as the `response_format` argument in your API call alongside the model and messages. The LLM will now always return an object with exactly those four keys, the correct types, and no extra fields.

The `description` field on each property is useful for complex questions — it gives the model extra guidance on how to populate the field. Real-world JSON schemas can run 600–700 lines for intricate pipelines; the structure scales.

A practical tip: keep all values as strings when you are doing simple extraction, because LLMs handle strings most reliably. When you need arithmetic (like `total_amount = price × quantity`), set the type to `number` and add the formula in the description so the model calculates it for you.

The rule of thumb: use JSON mode for quick experiments; use JSON schema for any production pipeline where downstream agents consume the output.

Validating the LLM's structured output

Even with a JSON schema in place, you should validate the response before passing it to the next agent. The schema reduces the failure rate dramatically, but the prevention-is-better-than-cure principle applies: always check the output programmatically.

A validation function does three things:

1. **Parse** — confirm the string is valid JSON with `json.loads`.
2. **Key check** — confirm every required key is present.
3. **Type check** — confirm each value has the expected type (e.g. `isinstance(data["price_per_unit"], (int, float))`).

You can add **business rule checks** on top: quantity should be greater than 0, total_amount should not be negative, and so on.

```
import json

def validate_product_response(raw_response):
    try:
        data = json.loads(raw_response)
    except json.JSONDecodeError:
        return None  # not valid JSON

    required_keys = ["product_name", "price_per_unit", "quantity", "total_amount"]
    if not all(k in data for k in required_keys):
        return None  # missing keys

    if not isinstance(data["price_per_unit"], (int, float)):
        return None  # wrong type
    if not isinstance(data["quantity"], (int, float)) or data["quantity"] <= 0:
        return None  # business rule violation

    return data  # all checks passed
```

If validation returns `None`, append feedback to the messages list and call the LLM again. The model sees the error, self-corrects, and usually returns a valid response on the next attempt. This retry loop is the safety net that makes structured output pipelines robust.

```
flowchart LR
    Prompt["Prompt"] --> LLM["LLM (JSON schema enforced)"]
    LLM --> Validate{"Validate output"}
    Validate -->|"Valid"| NextAgent["Next agent"]
    Validate -->|"Invalid"| Feedback["Append error feedback"]
    Feedback --> LLM
```

Structured tool responses

When an agent calls a **tool** (a Python function wired up to the LLM), the tool's return value goes back to the model as context. Returning a plain string is fine for tiny tools. When a tool does meaningful work — querying a CSV, calling an API, running a calculation — structured JSON makes the return value far more reliable for the model to interpret.

Use `json.dumps` to wrap everything the tool returns:

```
import json

def get_column_stats(column_name, data):
    try:
        values = [row[column_name] for row in data]
        result = {
            "status": "success",
            "column": column_name,
            "count": len(values),
            "mean": sum(values) / len(values),
            "min": min(values),
            "max": max(values)
        }
    except KeyError:
        result = {
            "status": "error",
            "message": f"Column '{column_name}' not found",
            "available_columns": list(data[0].keys())
        }
    return json.dumps(result)
```

Notice the `try / except` block. When something goes wrong, the tool returns a structured error object instead of raising an exception. The LLM reads the error message, understands what went wrong, and can decide to call a different tool or adjust its approach. This is called **graceful error return** — always return something the model can act on.

Idempotent tool design

Idempotency means that calling a function twice (or ten times) produces the same result and causes no additional harm. A weather lookup is naturally idempotent — calling it five times in a row just returns the same temperature.

Not all tools share this property. A **send payment** tool transfers money on every call. A **send email** tool fires off another email every time it runs. If an LLM calls these tools twice by mistake — something that can happen in multi-agent loops — the consequences are real: double charges, flooded inboxes.

The standard fix is a **request ID**. When the LLM decides to use a tool, it generates a unique ID for that specific tool call. Pass that ID into the tool function. The tool stores processed IDs; if it sees the same ID again, it skips re-execution and returns a success message immediately.

```
processed_orders = set()  # tracks completed request IDs

def place_order(product, quantity, request_id):
    if request_id in processed_orders:
        return json.dumps({"status": "success", "note": "already processed"})

    # execute the real order logic here
    processed_orders.add(request_id)
    return json.dumps({"status": "success", "product": product, "quantity": quantity})
```

This single pattern prevents a large class of production bugs. Tools that only read data (like weather lookups or database queries) do not need it. Tools that write, charge, or send something always do.

Watch out for: Forgetting idempotency on write tools is one of the most common bugs in multi-agent systems. If the LLM retries a failed tool call, a non-idempotent tool will execute the side effect again.

Putting it all together — the structured output pipeline

These four ideas form a complete, production-safe pipeline:

1. Set a low temperature (or zero) to reduce output randomness.
2. Apply a JSON schema to lock in the exact output shape.
3. Validate every LLM response before routing it to the next step.
4. Design tools to be idempotent (with request IDs for write operations).

The flow looks like this end-to-end:

```
flowchart TD
    UserQ["User question"] --> A1["Agent 1\n(JSON schema + low temp)"]
    A1 --> Val["Validate output"]
    Val -->|"Valid JSON"| A2["Agent 2\n(consumes structured data)"]
    Val -->|"Invalid"| Retry["Retry with error feedback → Agent 1"]
    A2 --> Tool["Tool call\n(idempotent, returns json.dumps)"]
    Tool --> A2
    A2 --> FinalOut["Final structured answer"]
```

JSON is the primary communication format across the entire stack — tool definitions, MCP configurations, API responses, and agent-to-agent handoffs all use JSON. Getting comfortable with JSON schema is therefore one of the highest-leverage skills in building reliable AI agents.

3. Key Takeaways

- LLMs are probabilistic models — the same prompt can return different text on every call, which makes free-text outputs unsafe for multi-agent pipelines.
- Setting `response_format` to `type: json_schema` with `strict: True` locks the LLM into returning exactly the fields, types, and structure you define — far more reliable than plain JSON mode.
- Always validate the structured response in code (parse → key check → type check → business rules) and loop back with feedback if something is wrong.
- Return tool outputs as `json.dumps(...)` objects so the LLM can reliably read results and errors; use a graceful error structure so the model can self-correct.
- Make write tools idempotent by tracking a request ID — if the same ID appears again, skip re-execution and return a success message immediately.

Mental model: Think of structured outputs as a customs declaration form at an airport. Every traveller fills in the same fields in the same boxes — regardless of what they carry — making it trivial for the officer on the other side to process each submission reliably.

Embeddings and semantic retrieval systems

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why keyword search fails and how embeddings fix it by comparing meaning instead of characters
- Convert text to vector embeddings using OpenAI's API and compute cosine similarity scores in Python
- Build a working semantic search function over a knowledge base
- Scale that system to large document collections using chunking and a vector database like ChromaDB

2. Detailed Explanation

Why keyword search fails — and what semantic search does differently

Imagine a 100-page company policy document with the sentence:

> "Employees may opt for remote work on the last working day of the week."

If a developer searches for "work from home", Ctrl+F returns zero results. Not because the policy doesn't cover it — it does — but because no individual character sequence matches. This is the fundamental limitation of **keyword search**: it matches characters, not meaning.

Semantic search — also called semantic retrieval — solves this by converting both the document and the query into numbers that capture meaning, then comparing those numbers. Before we get there, though, we need to understand why not just any number representation works.

One-hot encoding and why it falls short

Machine learning models cannot process raw text — they need numbers. **One-hot encoding** is the simplest way to convert a word to a number: give each word in a vocabulary its own slot in a list, put a 1 in that slot and 0 everywhere else.

The problem is immediate. The words "hi" and "hello" mean almost the same thing, yet one-hot encoding gives them completely unrelated vectors. There is no numerical closeness between them at all. For text-to-number conversion to be genuinely useful, similar meanings must produce similar numbers. That is exactly what embeddings deliver.

Embeddings — text converted to meaning-preserving vectors

An **embedding** is a fixed-length list of decimal numbers (a vector) produced from a piece of text, where semantically similar texts produce numerically close vectors.

Consider these two sentences:

- "I love dogs" → `[0.023, −0.041, 0.087, 0.012, …]` — 1,536 numbers
- "I love puppies" → `[0.025, −0.039, 0.091, 0.011, …]` — 1,536 numbers

Both sentences are about affection for young canines. The vectors are close — the distance between them in vector space is tiny.

Key facts to know:

Property	Detail
Output format	A single fixed-length vector per input string
Common dimensions	512, 720, 1,536, 3,072
OpenAI text-embedding-3-small	1,536 dimensions
OpenAI text-embedding-3-large	3,072 dimensions
Larger dimension	Captures more aspects of meaning

Think of each embedding as coordinates in a very high-dimensional space. Sentences about the same topic land near each other; sentences about completely different topics land far apart.

How embedding models learn

Embedding models are built on **transformer** architectures, which use a mechanism called **self-attention**. The foundational research paper — "Attention Is All You Need" — was published by Google researchers in 2017. That same architecture underpins ChatGPT, Gemini, and Claude.

During training the model sees vast amounts of text. One key technique is **masking**: a word in a sentence is hidden and the model must predict it. Through many iterations of this task, the model learns to encode semantic relationships into the vector space — the numbers it produces reflect what words and phrases actually mean.

The encoder component of the transformer is what produces the final embedding vector as output.

Choosing an embedding model — OpenAI vs open-source

You have two main paths:

Paid OpenAI models (API key required)

Model	Dimensions	Approximate cost
text-embedding-3-small	1,536	~$0.02 per 1 million tokens
text-embedding-3-large	3,072	Higher — see OpenAI pricing

Open-source models via Hugging Face

Hugging Face is a platform hosting thousands of machine learning models, datasets, and spaces. A search for "text embeddings" returns hundreds of free options — for example, `Zena-Embedding-V5-text-small`, `GTE-text-nano`, `GTE Multilingual Embedding`, and Qwen2-based

models. Install the `transformers` library (`pip install transformers`) and run them locally with no API key.

The trade-off: open-source models are free but take time to download and consume local compute. In practice, companies use trusted paid providers such as OpenAI or Anthropic approximately 90% of the time.

Here is how to call the OpenAI embedding API:

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

def get_embeddings(text):
    response = client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding
```

This function accepts any string and returns a 1,536-dimensional vector you can use downstream.

Cosine similarity — measuring how close two vectors are

Cosine similarity is a score that tells you how semantically close two vectors are. It is derived from the angle between the two vectors when drawn from the origin. A small angle means the vectors point in the same direction — and therefore represent similar meaning.

The formula:

```
cosine_similarity = dot_product(v1, v2) / (norm(v1) * norm(v2))
```

In NumPy:

```
import numpy as np

def cosine_similarity(vector1, vector2):
    v1 = np.array(vector1)
    v2 = np.array(vector2)
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
```

Interpreting the score:

Score	Meaning
= 1.0	Identical meaning
> 0.8	Very similar
0.7 – 0.8	Related but not identical
0.5 – 0.7	Loosely related
< 0.5	Probably unrelated

Here is the score in action over a set of sentences, all compared against "I love playing cricket":

Sentence	Score	Why
"Cricket is my favorite sport"	~0.80+	Same topic
"I enjoy watching football matches"	Moderate	Sports, different game
"Python is a programming language"	~0.26	Unrelated
"The stock market crashed today"	~0.14	Completely different domain

Watch out for: Cosine similarity measures topical closeness, not factual correctness. "Earth is flat" and "Earth is round" score approximately 0.81 because they are both about the shape of the Earth — they are not both true. This mirrors the distinction between correlation and causation: two things can be correlated without either causing the other.

Building a semantic search system

Now we can combine embeddings and cosine similarity into a working search system. The approach has four steps:

```
flowchart TD
    StoreDoc["Embed every document chunk\nand store the vectors"] --> UserQuery["User submi
    UserQuery --> EmbedQuery["Convert the query\nto an embedding"]
    EmbedQuery --> Rank["Compute cosine similarity\nagainst every stored vector"]
    Rank --> Return["Return top-N results\nsorted by score"]
```

In code, the search function looks like this:

```
kb_embeddings = {}
for doc in knowledge_base:
    kb_embeddings[doc] = get_embeddings(doc)

def search_notes(query, top_k=3):
    query_embedding = get_embeddings(query)
    scores = []
    for doc, emb in kb_embeddings.items():
        score = cosine_similarity(query_embedding, emb)
        scores.append((score, doc))
    scores.sort(reverse=True)
    return scores[:top_k]
```

The knowledge base here contains short descriptions of machine learning algorithms. Running the search:

- Query: "What algorithm uses multiple trees?" → top hit: "Random forest combines many decision trees and lets them vote." (similarity ~0.55)
- Query: "How do I prevent my model from memorizing the data?" → top hit: "Overfitting happens when a model memorizes the data." (similarity ~0.48)

No exact keywords match either query — yet the correct answer surfaces each time, because embeddings capture meaning. This is the core value of a semantic retrieval system over traditional keyword lookup.

Why this approach breaks down at scale — and the two solutions

The search function above re-computes embeddings for the entire knowledge base every time a query arrives. That is fine for a handful of lines. Scale up to a 50-page PDF and the problems multiply fast:

- Embedding every line at query time is too slow.
- Recomputing all embeddings on every question is wasteful.
- Storing thousands of 1,536-dimensional vectors in a plain SQL table and scanning every row for similarity introduces high latency.

Two components solve these problems together and make a production-grade retrieval system practical: **document chunking** and **vector databases**.

Document chunking

Chunking splits a large document into smaller, independently embeddable segments so that embeddings can be computed once and stored persistently — never recomputed per query. This pre-computation is what makes retrieval fast at scale.

A simple strategy splits on paragraph boundaries and keeps only segments with at least 30 words:

```
def chunk_by_paragraph(document, min_words=30):
    paragraphs = document.split("\n")
    chunks = []
    for para in paragraphs:
        para = para.strip()
        if len(para.split()) >= min_words:
            chunks.append(para)
    return chunks
```

To put the scale in perspective: a 50-page document with three chunks per page produces 150 chunks. All 150 embeddings are computed once and stored. Queries then search over those stored vectors — no recomputation.

Vector databases and HNSW

A **vector database** is a database built specifically for storing and searching high-dimensional embedding vectors efficiently. It is the storage layer of any production semantic retrieval system.

A regular SQL table can technically hold 1,536-column rows, but similarity-searching across hundreds of thousands of such rows at query time is slow. Vector databases replace that brute-force scan with a specialized index.

The most widely used indexing algorithm is **HNSW (Hierarchical Navigable Small World)**. HNSW organizes the vector space as a layered graph, allowing approximate nearest-neighbor lookups that are very fast even over millions of vectors.

Popular vector databases:

Database	Notes
ChromaDB	Open-source, free, handles embeddings internally
FAISS	Open-source library by Meta
Pinecone	Managed cloud service
Weaviate	Open-source with cloud option
Milvus	Open-source, suited for large-scale deployments
pgvector	PostgreSQL extension for vector storage
AWS-managed options	Preferred in AWS ecosystems for native integration

Companies running on a cloud platform tend to use that platform's native vector database because it integrates directly with the rest of their infrastructure.

ChromaDB in practice

ChromaDB is open-source and free. Its standout feature is that it handles embedding internally using the `all-MiniLM-L6-v2` model by default — you do not need to call a separate embedding function.

```
import chromadb

chroma_client = chromadb.Client()
collection = chroma_client.create_collection(
    name="ml_notes",
    metadata={"description": "Machine learning study notes"}
)

chunks = chunk_by_paragraph(document)
collection.add(
    documents=chunks,
    ids=[f"chunk_{i}" for i in range(len(chunks))]
)
```

Querying is equally concise:

```
results = collection.query(
    query_texts=["How do I prevent overfitting?"],
    n_results=3
)
```

ChromaDB returns results sorted by **distance** (not similarity score). Distance and similarity are inversely related: a small distance means high similarity. Metadata such as page number, source document name, and chunk ID can be stored alongside each vector to enable filtered retrieval.

LangChain — the unifying framework

LangChain is a framework that unifies embedding models, vector databases such as ChromaDB, and LLM clients into a single API. It removes the boilerplate of wiring each integration separately.

Understanding the underlying components makes it possible to use LangChain effectively and to debug retrieval problems when they arise. Some companies use LangChain; others use frameworks such as Amazon Strands (an open-source agent SDK from AWS) depending on their ecosystem. The underlying concepts of embedding and retrieval remain the same regardless of which framework sits on top.

3. Key Takeaways

- **Embeddings** convert text into fixed-length vectors of decimal numbers, where semantically similar text produces numerically close vectors — fixing the keyword-search problem that tools like Ctrl+F cannot solve.
- **Cosine similarity** gives a score between 0 and 1 based on the angle between two vectors; scores above 0.8 indicate very similar meaning, but high similarity does not guarantee factual agreement.
- A semantic search system embeds every document chunk once, then at query time embeds the query and returns the highest-scoring chunks — no keyword overlap needed.
- **Chunking** splits large documents into storable segments (using strategies like minimum 30-word paragraphs) so embeddings are computed once, not re-computed per query.
- **Vector databases** (ChromaDB, FAISS, Pinecone, and others) use HNSW indexing to search millions of vectors quickly — far faster than scanning a plain SQL table row by row.

Mental model: Think of embeddings as GPS coordinates for meaning. Sentences about the same idea cluster in the same neighborhood of a very large map. Semantic search finds the nearest neighbors to your query's coordinates.

Retrieval-Augmented Generation (RAG) Basics

1. What You'll Learn in This Section

This lesson covers the basics of Retrieval-Augmented Generation — what it is, why it matters, and how to build one from scratch. By the end, you'll be able to:

- Explain what RAG is and why it solves real limitations of bare LLM API calls
- Build a two-phase RAG pipeline — indexing documents offline and retrieving relevant chunks per query
- Compare chunking strategies and identify why chunk size and top-k are tunable hyperparameters
- Distinguish RAG variants (Naive, Advanced, Agentic, CAG, and others) and describe when each applies

2. Detailed Explanation

What RAG Is and Why It Exists

Retrieval-Augmented Generation (RAG) is a design pattern that augments an LLM's response by fetching relevant pieces of text at query time. Rather than sending an entire document collection to the model, the system retrieves only the small chunks relevant to the user's question. It combines those chunks with the question in a prompt and lets the LLM produce a factual answer from that context.

Three practical problems make RAG necessary when working with private or large document sets:

1. **LLMs do not know private data.** A model like GPT-4o has no access to a company's HR policies, internal budgets, or product knowledge bases. Without access, it either says it does not know or produces a **hallucination** — a confident but factually incorrect answer.
2. **Context windows are finite and expensive.** GPT-4o supports up to 128,000 tokens in its context window, and input tokens cost about $2.50 per million. Sending thousands of pages on every query is impractical for hundreds of concurrent users.
3. **Sensitive data must stay private.** Uploading confidential documents to an external provider (OpenAI, Google Gemini, Anthropic Claude) exposes proprietary information. RAG keeps the full document corpus on-premise; only small retrieved chunks ever reach the LLM.

Consider a company that stores thousands of internal documents — HR policies, product knowledge bases, compliance guides. A chatbot built with bare LLM calls would either send everything (massively expensive and a privacy risk) or know nothing. RAG splits the problem: index the documents once offline, then on each query retrieve only the 3–10 most relevant chunks and send those to the model. Cost drops dramatically and data privacy is preserved.

Embeddings and Cosine Similarity — The Foundation

Before a chunk of text can be compared against a query, it must be converted to a number that a computer can compare. An **embedding** is a numerical vector representation of text that captures its semantic meaning. Texts that mean similar things produce vectors that are close together in vector space.

Cosine similarity measures how similar two vectors are in direction, regardless of their magnitude. A higher score indicates greater semantic similarity. This is the comparison engine that lets RAG find the chunks most relevant to a user's question.

ChromaDB uses the **all-MiniLM-L6-v2** sentence-transformer model as its default embedding model. It produces compact, dense vectors and was fine-tuned on approximately 1 billion sentence pairs. One important constraint: it truncates input text longer than 256 words. Chunks should stay below that threshold to avoid silent data loss.

Watch out for: The query embedding and the document chunk embeddings must be produced by the same model. If you switch embedding models after indexing, the vectors live in different spaces and cosine similarity scores become meaningless.

The Indexing Pipeline — Building the Knowledge Store

The **indexing pipeline** runs once (and is re-run only when documents are added or updated). It turns raw files into a searchable vector database.

```
flowchart LR
    A[Raw Documents] --> B[Chunking]
    B --> C[Embedding\nall-MiniLM-L6-v2]
    C --> D[Vector Database\nChromaDB]
    D --> E[Stored: text + id + metadata]
```

Step 1 — Load documents. Collect all source files. As an example, two text files are used: `company_hr_policy.txt` (7,464 characters) and `product_knowledge_base.txt` (about 9,800 characters).

Step 2 — Chunk the documents. Each file is split into smaller, self-contained text segments. The example implementation chunks at paragraph boundaries (double newlines) and discards any chunk shorter than 50 characters. This produces roughly 25 chunks from the HR policy and about 54 from the product knowledge base — 79 chunks total.

Several chunking strategies are available:

Strategy	How it splits
Paragraph-level	At double newlines; simplest approach
Line-by-line	At every newline; generally not recommended — destroys context
Fixed-size	Every N characters or tokens; can cut mid-sentence
Sliding-window	Fixed-size with overlap so adjacent chunks share text
Sentence-based	At sentence boundaries
Semantic	At natural topical break points
Recursive	Progressively finer separators until chunks reach a target size (LangChain's `RecursiveCharacterTextSplitter`)
Hierarchical	Maintains parent/child chunk relationships

Chunk size and overlap are **hyperparameters** — tunable values with no universally correct setting. They must be chosen experimentally for each dataset.

Step 3 — Embed the chunks. Each chunk is passed through the embedding model. ChromaDB applies all-MiniLM-L6-v2 automatically when you add documents.

Step 4 — Store in a vector database. Each chunk is stored with three fields: the raw text (`document`), a unique identifier (`id`), and **metadata** — a dictionary recording the source file name (e.g., `{"source": "HR Policy"}`). Once all 79 chunks are loaded, the in-memory ChromaDB instance uses approximately 79.3 MB.

```
import chromadb

chroma_client = chromadb.Client()
collection = chroma_client.create_collection(name="company_docs")

collection.add(
    documents=documents,
    ids=ids,
    metadatas=metadatas
)
```

The Retrieval Pipeline — Answering a Query

The **retrieval pipeline** runs on every user query. It converts the question to a vector, finds the best-matching chunks, builds a prompt, and calls the LLM.

Step 1 — Embed the question. The same embedding model converts the user's question into a vector.

Step 2 — Find the top-k most similar chunks. ChromaDB runs a cosine similarity search and returns the `n_results` closest chunks. The default in the example is k = 3.

```
def retrieve(question, n_results=3):
    results = collection.query(
        query_texts=[question],
        n_results=n_results
    )
    return results["documents"][0], results["metadatas"][0]
```

Step 3 — Build a prompt. Retrieved chunks are concatenated into a `context` string. The system message instructs the LLM to answer only from the provided context and to say "I don't have enough context or information to answer this question" when the context is insufficient.

Step 4 — Call the LLM. The example uses the OpenAI API with `gpt-4o-mini`. Open-source models via Ollama or Groq are also valid. Because the LLM is doing extraction from already-retrieved text rather than knowledge recall, less powerful models often suffice.

```
from openai import OpenAI

client = OpenAI(api_key=OPENAI_API_KEY)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    temperature=0
)
answer = response.choices[0].message.content
```

Step 5 — Return the grounded answer. The **grounded answer** is traceable to specific retrieved chunks and source documents. Because metadata records which file each chunk came from, the calling code can surface the source document name alongside the answer for cross-verification.

The pipeline produces concrete results. For example, asking "What is the work from home policy?" returns that employees who have completed their six-month probation are eligible for up to two work-from-home days per week. Asking "What is the capital of France?" returns "I don't have enough context" — illustrating that the LLM's behaviour is controlled entirely by the prompt, not by hardcoded logic.

Watch out for: When `n_results` is set too high (e.g., 10), cosine similarity can surface topically unrelated chunks. The system retrieved product knowledge base chunks in response to an HR-policy question when k was increased to 10. The optimal top-k must be determined experimentally.

Types of RAG

The basic pipeline above is called **Naive RAG** — chunk, embed, store, retrieve top-k, prompt the LLM, return the answer. It is a valid production starting point, but several variants address its limitations:

Variant	What it adds or changes
Advanced RAG	Query re-writing, chunk re-ranking, and chunk compression on top of the naive pipeline
Graph RAG	Organises the knowledge base as a graph; enables reasoning over entity relationships
Agentic RAG	Wraps retrieval as an LLM tool; the LLM decides when and whether to call it
Modular RAG	Components (chunker, retriever, reranker, generator) can be swapped independently
Iterative RAG	Repeats retrieve-and-generate multiple times, using each output to refine the next query
Adaptive RAG	Selects the retrieval strategy dynamically based on question type
Hierarchical RAG	Maintains chunks at multiple granularities and retrieves at the appropriate level
Multi-agent RAG	Multiple specialised agents each handle a subset of retrieval or reasoning
CAG (Cache-Augmented Generation)	Caches previously retrieved chunk sets; reuses them when a new question is ≥ ~95% similar to a cached one — much faster for repeat queries
Multimodal RAG	Extends RAG to images, tables, video, and other non-text content

Across all variants, the core indexing-then-retrieval concept is the same — the retrieval-augmented approach simply adds or modifies specific components to address quality, speed, or data-type limitations.

Agentic RAG in Detail

In the basic pipeline, retrieval always fires. In **agentic RAG**, the retrieve function is registered as a tool in the LLM's tool-call schema. The LLM decides whether to call it and with what arguments.

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "retrieve",
            "description": "Retrieve relevant document chunks for a given question.",
            "parameters": {
                "type": "object",
                "properties": {
                    "question": {"type": "string"},
                    "n_results": {"type": "integer", "default": 3}
                },
                "required": ["question"]
            }
        }
    }
]
```

This makes the system more flexible. The LLM can skip retrieval for questions it can answer directly from general knowledge, and it can call retrieval multiple times with different query formulations for complex questions.

The **Model Context Protocol (MCP)** is a standard for exposing collections of tool definitions to an LLM — the same concept as a single tool call, but packaged for reuse across many clients.

Evaluating and Tuning a RAG System

There is no single automated metric that reliably evaluates a RAG pipeline. A practical manual approach involves six steps:

1. Create a question-and-answer dataset — 100 to 1,000 questions whose answers are known to exist in the documents.
2. Run the RAG pipeline on every question and compare the produced answer to the expected one.
3. Check consistency — ask the same question multiple times and verify the system returns consistent answers.
4. Check factualness — verify answers cite real information from the documents rather than hallucinated facts.
5. Measure latency — record how long each query takes and profile slow calls.

6. Tune hyperparameters — if answers are incomplete, increase top-k; if answers are noisy, decrease top-k or improve chunking.

This process is inherently manual. Evaluating and tuning a production-grade RAG system typically takes six months or more, depending on data volume, data collection effort, and team size.

Production Challenges

Challenge	Description
Poor retrieval quality	Paragraph chunking can split related information across two chunks; a query may match only one, leaving the answer incomplete. Overlapping chunks, a larger top-k, and re-ranking help.
Wrong top-k value	Too small and the relevant chunk may not be retrieved. Too large and the LLM's context fills with noise, causing it to ignore the relevant chunks.
LLM ignoring context	The LLM sometimes answers from its own parametric knowledge rather than retrieved context, especially when the context is long. Strong system-prompt instructions mitigate this.
Hallucination residual	Even with RAG, if retrieval misses the relevant chunk, the LLM may hallucinate. "I don't know" fallback instructions reduce the damage.
Cold start problem	When the document corpus is new or sparse, similarity search yields low-quality results. The system needs enough high-quality indexed content to perform well.
Latency at scale	With millions of chunks, scanning all vectors becomes too slow. Approximate nearest-neighbour indexes — **HNSW (Hierarchical Navigable Small World)** and **IVF (Inverted File Index)** — are built into vector databases to maintain sub-second query times at scale.
Data collection effort	Building a company-wide RAG system requires gathering documents that may not yet exist in digital form. Stakeholder interviews and document creation can dominate the project timeline.

LangChain and the From-Scratch Approach

LangChain is a Python framework that provides pre-built components for LLM applications — document loaders, text splitters (including `RecursiveCharacterTextSplitter`), and vector-store integrations. Building the retrieval-augmented pipeline from scratch first — using only `os`, `json`, `openai`, and `chromadb` — helps you understand what each component does before reaching for convenience wrappers. Mastering these basics before adopting a framework prevents confusion when something breaks in production.

3. Key Takeaways

- RAG retrieves only the small, relevant chunks that answer a user's question and passes them to the LLM — solving the private-data, cost, and privacy problems of using LLMs on proprietary document sets.
- The pipeline has two phases: an offline **indexing phase** (load → chunk → embed → store) and a per-query **retrieval phase** (embed question → cosine search → build prompt → call LLM → return grounded answer).
- Chunk size, chunk overlap, and top-k are hyperparameters with no universal correct values. They must be tuned experimentally for each dataset — getting them wrong degrades both recall and precision.
- Naive RAG is a valid starting point; understanding its basics is essential before adopting variants like Advanced RAG, Agentic RAG, and CAG, which each address specific limitations around quality, flexibility, and speed.
- Evaluating a production RAG system is a manual, time-consuming process that typically takes six months or more — there is no single automated metric that reliably measures pipeline quality.

Mental model: Think of RAG as a librarian who reads the question, walks to the right shelf, pulls out three relevant pages, and hands them to the LLM — rather than dumping the entire library on the model's desk.

Productionizing a RAG chatbot with FastAPI and Hugging Face Spaces

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why a notebook-based RAG chatbot must become an API before real users can reach it
- Build a FastAPI application with GET and POST endpoints, input validation via Pydantic, and auto-generated documentation
- Assemble the full production stack — FastAPI, Uvicorn, Pydantic, logging, and environment variables — into a working `main.py`
- Deploy the application for free on Hugging Face Spaces using a Docker-based workflow

2. Detailed Explanation

From notebook to production: why the move matters

A RAG chatbot that lives inside a Google Colab or Jupyter notebook works beautifully — on the machine where the notebook is running. Nobody outside that machine can query it. **Productionizing** means taking code that works locally and making it accessible, scalable, and maintainable for real users. Each step in this section addresses a specific productionizing concern: API exposure, input validation, logging, secret management, and deployment.

The solution is to expose the chatbot's logic as an **API (Application Programming Interface)** — the standard way software communicates with other software. Once it's an API, any client (web app, mobile app, or service) can send a question over HTTP and get an answer, without touching the underlying code or machine.

Here is the overall journey at a glance:

```
flowchart TD
    Notebook["RAG chatbot\n(Jupyter notebook)"] --> FastAPI["FastAPI wraps logic\nas HTTP e
    FastAPI --> Uvicorn["Uvicorn runs the app\n(localhost:8000)"]
    Uvicorn --> Docker["Docker packages\nthe entire environment"]
    Docker --> HFSpaces["Hugging Face Spaces\nhosts it publicly"]
    HFSpaces --> PublicURL["Public URL\nhttps://user-space.hf.space"]
```

Each layer in this diagram solves one specific problem. The sections below explain each layer in depth.

FastAPI: turning Python functions into HTTP endpoints

FastAPI is a modern Python web framework that converts ordinary Python functions into HTTP API endpoints with minimal extra code. It provides automatic input validation and auto-generates interactive documentation — no manual documentation work required.

The key mechanism is the **decorator** — a line starting with `@app.get(...)` or `@app.post(...)` that you place above a function. The decorator registers that function as an endpoint at the specified URL path.

```
from fastapi import FastAPI

app = FastAPI(title="RAG Chatbot", version="1.0")

@app.get("/")
def home():
    return {"status": "running", "message": "Novatech Chatbot Live"}

@app.get("/greet")
def greet(name: str):
    return {"message": f"Hello {name}"}

@app.get("/add")
def add(a: int, b: int):
    return {"result": a + b}
```

In this example, `@app.get("/")` registers `home()` as the endpoint at the root path `/`. The `/greet` and `/add` endpoints each accept parameters that FastAPI maps automatically from the URL.

GET vs POST

Two HTTP methods are used throughout this project:

Method	When to use	Where parameters live
GET	Retrieving data	In the URL (query string)
POST	Sending data to the server	In the request body

The RAG chatbot's question-answering endpoint uses **POST** because the client sends a question payload to the server. GET endpoints, like `/greet` and `/add`, pass their parameters in the URL:

```
http://localhost:8000/add?a=10&b=25
```

The `?` separates the path from the query string. `&` separates individual parameters. FastAPI maps `a` and `b` automatically and returns `{"result": 35}`.

Auto-documentation at `/docs`

Navigate to `http://localhost:8000/docs` in any browser while the server is running. FastAPI generates an interactive UI listing every endpoint, its expected parameters, its expected response, and a "Try it out" button. This is produced automatically — no extra code needed.

Uvicorn: the server that opens the door

FastAPI defines what the application does, but by itself it cannot listen for incoming HTTP requests. **Uvicorn** is the **ASGI server** that runs the FastAPI application and makes it reachable over a network.

Think of FastAPI as the chef and kitchen — the actual logic and recipes. Uvicorn is the restaurant itself. Without opening the restaurant, no customers can reach the chef even if everything in the kitchen is ready.

Start the server with one command:

```
uvicorn main:app --reload
```

- `main` — the Python filename (without `.py`).
- `app` — the FastAPI instance inside that file.
- `--reload` — automatically restarts the server when the code changes.

The server starts on `http://localhost:8000` by default. Stopping it (Ctrl+C or `pkill -f uvicorn`) immediately makes all endpoints unreachable.

Pydantic: catching bad data before it reaches your code

Pydantic is a Python library for data validation. It enforces data types at runtime using **models** — classes that inherit from `BaseModel` and describe required fields and their types.

Without Pydantic, a function expecting integers still accepts strings and only fails with a generic `TypeError` deep inside the logic — difficult to debug. With Pydantic, the error is caught before the function runs and returned as a structured **ValidationError** that names the failing field and explains why:

```
from pydantic import BaseModel

class AddRequest(BaseModel):
    a: int
    b: int

def add(request: AddRequest):
    print(request.a + request.b)

add(AddRequest(a=5, b=6))           # Works — prints 11
add(AddRequest(a="hello", b=6))     # ValidationError: "Input should be a valid integer"
```

The error message names the field (`a`), states the expected type (`int`), and shows the actual value received (`"hello"`). This makes debugging straightforward even in large codebases.

Watch out for: type hints alone don't enforce validation. FastAPI only checks types for simple GET endpoints when a Pydantic model is the parameter type. Without Pydantic, wrong types still slip through.

Pydantic models in the RAG API

Two models define the shape of the chatbot's traffic:

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    question: str

class ChatResponse(BaseModel):
    answer: str
    sources: list
    request_id: str
```

Every POST to `/chat` must contain a `question` string — `ChatRequest` enforces this. Every response returns an `answer` , a `sources` list, and a `request_id` — `ChatResponse` defines that shape.

Logging and environment variables: production hygiene

Two habits separate notebook code from productionized code: structured logging and secret management.

Logging replaces `print` statements. The `logging` standard library module records what is happening inside the running server with timestamps and severity levels. Configure it once at startup and use `logging.info(...)` throughout:

```
import logging
logging.basicConfig(
    format="%(asctime)s - %(levelname)s - %(message)s",
    level=logging.INFO
)
```

Every time a user sends a question, the server logs it automatically. This audit trail is invaluable when diagnosing issues in a live system.

Environment variables keep API keys out of source files. Sensitive credentials — like `GROQ_API_KEY` and `HF_TOKEN` — are stored in a `.env` file and loaded at runtime using the `python-dotenv` package:

```
from dotenv import load_dotenv
load_dotenv()
```

The `.env` file is never committed to version control. This means the credentials stay off GitHub and off Hugging Face — the running server reads them from its environment instead.

Assembling main.py: the full RAG API

With all the pieces understood individually, they come together in a single file called `main.py`. The application also relies on a **virtual environment** — an isolated Python installation created with `python3 -m venv .venv` — to keep project dependencies separate from the system Python.

The required packages are listed in `requirements.txt`:

```
pip3 install fastapi openai chromadb python-dotenv uvicorn pydantic
```

The `openai` package is used here not to call OpenAI but to call the **Groq API**, which provides an OpenAI-compatible client. The Groq API key is stored in the `.env` file as `GROQ_API_KEY`.

Inside `main.py`, four things happen in order:

1. Imports and configuration — `load_dotenv()` loads secrets, `logging.basicConfig(...)` sets up logging, the FastAPI `app` is created.
2. `load_and_index_data()` — reads `.txt` documents from the `data/` folder, chunks them, and inserts the chunks into a ChromaDB collection named `docs`. This runs once at startup and produced 120 chunks against the sample documents.
3. `ask_rag(question)` — the core RAG function: queries ChromaDB for relevant chunks, builds a prompt, calls the Groq LLM, and returns the answer string.
4. Two endpoints — GET `/` (health check) and POST `/chat` (the main chatbot endpoint).

```
from fastapi import FastAPI
from pydantic import BaseModel
import logging
from dotenv import load_dotenv

load_dotenv()
logging.basicConfig(format="%(asctime)s - %(levelname)s - %(message)s", level=logging.INFO)

app = FastAPI(title="RAG Chatbot", version="1.0")

class ChatRequest(BaseModel):
    question: str

class ChatResponse(BaseModel):
    answer: str
    sources: list
    request_id: str

@app.get("/")
def home():
    return {"status": "running", "message": "Novatech Chatbot Live"}

@app.post("/chat")
def chat(request: ChatRequest) -> ChatResponse:
    logging.info(f"User question: {request.question}")
    answer = ask_rag(request.question)
    return ChatResponse(answer=answer, sources=[], request_id="...")
```

Run it locally with `uvicorn main:app --reload`. The server starts at `http://localhost:8000` and logs HTTP status codes: `200` for success, `404` for missing paths, and `422` when validation fails.

The knowledge base is stored as `.txt` files in a `data/` folder (security policy, work-from-home policy, password policy, and others). The RAG correctly limits its answers to indexed content — asking "Who is Donald Trump?" returns "I don't have enough information about this."

The project file structure keeps everything organised:

```
fast-api/
├── app/
│   ├── main.py
│   ├── simple_demo.py
│   ├── requirements.txt
│   ├── Dockerfile
│   └── data/
├── .env
└── .venv/
```

Docker and Hugging Face Spaces: going public

The local server only works while a developer's machine is on. Serving real users requires running the application on external infrastructure. Two technologies make this possible: Docker and Hugging Face Spaces.

Docker packages the entire execution environment — Python version, installed packages, application code, and data files — into a portable **Docker image** that runs identically anywhere. A **Dockerfile** is the recipe for building that image. Without it, a remote server has no way to know what environment the application needs. Docker eliminates "it works on my machine" failures.

Hugging Face Spaces is the free-tier hosting option. Hugging Face is an open-source platform for machine learning models, datasets, and applications. Its Spaces feature lets anyone deploy a web app or API for free using Hugging Face's own infrastructure. Every deployed Space receives a public URL in the form:

```
https://<username>-<space-name>.hf.space
```

Anyone with that URL can call the API. This makes Spaces the free equivalent of paid cloud hosting (such as AWS EC2 or ECS) for AI applications.

The Dockerfile for this project instructs Hugging Face exactly how to build the environment:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY main.py .
COPY data/ ./data/
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

Key points: - `python:3.11-slim` is a small base image with Python but no unnecessary system packages. - The `COPY` and `RUN pip install` steps install all dependencies inside the container. - The `CMD` runs Uvicorn on port `7860` — the default Hugging Face Spaces port — and on `0.0.0.0` so the container accepts external connections.

The deployment workflow

Step	What happens
1. Create a Hugging Face account and open Spaces	—
2. Create a new Space	Name it (e.g., `novatech-project`), select "Docker" as the SDK, set visibility to public
3. Generate an access token	Settings → Tokens → New token with write permission; save as `HF_TOKEN` in `.env`
4. Clone the Space	`git clone https://huggingface.co/spaces/<username>/<space-name>`
5. Copy application files	Copy `main.py`, `requirements.txt`, `Dockerfile`, and `data/*.txt` into the cloned folder
6. Commit and push	`git add .` → `git commit -m "Deploying RAG chatbot with FastAPI"` → `git push origin main`
7. Add secrets in Space settings	Space → Settings → Variables and Secrets → New Secret: `GROQ_API_KEY`
8. Hugging Face builds automatically	The build log shows: build image → export cache → push → app startup
9. Public URL goes live	e.g., `https://your-username-novatech-project.hf.space/docs`

Secrets are added through the Space settings UI — never stored in the repository. The running container reads `GROQ_API_KEY` from its environment, exactly as `load_dotenv()` does locally.

Once live, calling POST `/chat` with `{"question": "What is the password security details?"}` returns:

```
{
  "answer": "Passwords must be at least 12 characters with at least one uppercase letter.",
  "sources": [...],
  "request_id": "..."
}
```

Optional: adding a Streamlit front end with Nginx

After the API is live, a browser-based chat interface can be layered on top using **Streamlit** — a Python library for building browser applications quickly. The Streamlit app sends POST requests to the `/chat` endpoint and displays the answers in a chat interface, with `role: user` and `role: assistant` message objects.

When both FastAPI and Streamlit run inside the same Space, an **Nginx** reverse proxy routes traffic. Browser requests go to Streamlit on one port; API calls go to FastAPI on another. The Dockerfile is updated to add Nginx and Streamlit, copy `streamlit_app.py`, and start both services alongside Uvicorn.

3. Key Takeaways

- A notebook-based RAG chatbot must become an API before anyone outside the developer's machine can use it; FastAPI is the framework that makes the conversion nearly effortless with decorators.
- FastAPI, Uvicorn, and Pydantic work as a trio: FastAPI defines the endpoints, Uvicorn runs the server, and Pydantic validates every incoming request before it reaches the application logic.
- Production code keeps API keys in a `.env` file (loaded by `python-dotenv`) and uses `logging` instead of `print` — essential for debugging live systems.
- Docker packages the entire environment into a portable image, eliminating the environment mismatch that is one of the last barriers to productionizing any Python application.
- Hugging Face Spaces hosts Docker apps for free and provides a public URL — the lowest-barrier path from local API to live service.

Mental model: Productionizing is like opening a restaurant. FastAPI writes recipes, Pydantic enforces the menu, Uvicorn opens the restaurant locally, Docker ships the kitchen to a new city, and Hugging Face Spaces provides the building for free.

Orchestration and Agent Workflow Design: Building Multi-Step LLM Pipelines with LangGraph

What You Will Learn

- Explain what LangGraph is and why orchestration frameworks exist on top of raw LLM calls.
- Identify the core building blocks of a LangGraph workflow — nodes, edges, and state — and trace how data flows between them.
- Design a retry mechanism inside an agent workflow so failed steps recover gracefully instead of crashing the entire pipeline.
- Build a simple end-to-end agent flow that combines nodes, conditional transitions, and retry logic into a working design.

1. Why Orchestration? The Problem with One Big Prompt

A single chef cooking a five-course meal alone, from scratch, with no recipe cards and no sequence — chopping, boiling, plating all happening in their head at once — eventually drops something. A professional kitchen instead breaks the meal into stations: prep station, grill station, plating station, each with a clear handoff to the next. Nothing gets lost, and if the grill station runs out of butter, the kitchen has a backup plan rather than the whole meal collapsing.

A single LLM call asked to "read this document, extract the key facts, verify them, and write a report" is the lone chef. It works for small tasks, but as tasks grow more complex — multiple steps, conditional logic, tool calls, error recovery — cramming everything into one prompt becomes fragile and hard to debug.

Orchestration is the practice of breaking a complex LLM-powered task into smaller, well-defined steps, then explicitly controlling how those steps connect, branch, and recover from failure. **LangGraph** is a framework built specifically for this — it lets you define your workflow as a graph, where each step is a node and the connections between steps are edges.

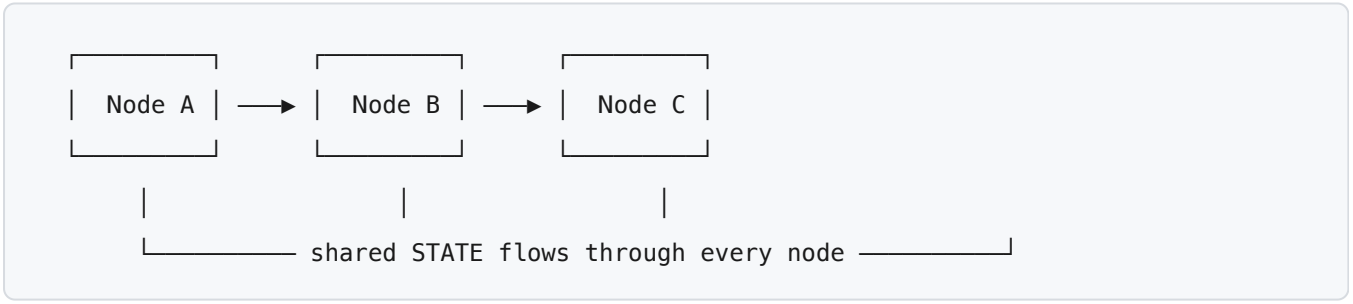
```
ONE BIG PROMPT                    ORCHESTRATED WORKFLOW

"Do everything at once"           Step 1: Extract → Step 2: Verify
      │                                        ↓ (if invalid)
      ▼                                 Step 2b: Retry
Single fragile output             Step 3: Summarize → Output
                                   Each step is inspectable, retryable,
                                   and independently testable.
```

---

2. LangGraph Basics: Thinking in Graphs, Not Scripts

A regular Python script runs top to bottom, one line after another. A **graph-based workflow** thinks differently — instead of a straight line, you have a network of steps where the next step depends on what just happened.

LangGraph models a workflow as a **directed graph**: a set of **nodes** (the individual steps of work) connected by **edges** (the paths that decide which node runs next). On top of this, LangGraph maintains a **state** — a shared object that gets passed from node to node, carrying whatever information has been gathered so far.



Think of state like a clipboard passed between departments in an office. Each department reads what's already on the clipboard, adds their own notes, and passes it along. By the time it reaches the last department, the clipboard contains the accumulated work of everyone before it — nobody has to start from scratch.

A minimal LangGraph workflow has three required pieces:

```
from langgraph.graph import StateGraph, END

# 1. Define the state — what data flows through the graph
class AgentState(dict):
    input: str
    output: str

# 2. Define nodes — functions that read and update state
def process_node(state):
    state["output"] = f"Processed: {state['input']}"
    return state

# 3. Build the graph
graph = StateGraph(AgentState)
graph.add_node("process", process_node)
graph.set_entry_point("process")
graph.add_edge("process", END)

app = graph.compile()
result = app.invoke({"input": "hello"})
```

Every node is just a function — it receives the current state, does some work (which could be a plain computation, a tool call, or an LLM call), and returns the updated state. The graph's job is to decide which node runs next.

---

3. Nodes and Transitions: The Building Blocks

Nodes — the units of work

A **node** represents one discrete step in your workflow. It could call an LLM, run a calculation, query a database, or call an external API. The defining property of a good node is that it does one job and does it predictably — much like a function in well-structured code.

```
  Good node design:                 Poor node design:

  "summarize_text"                  "do_everything"
  - takes text                      - extracts, summarizes, verifies,
  - returns a summary                 formats, and emails — all at once
  - one clear responsibility        - impossible to debug or retry
                                       a single piece
```

Edges — the transitions between nodes

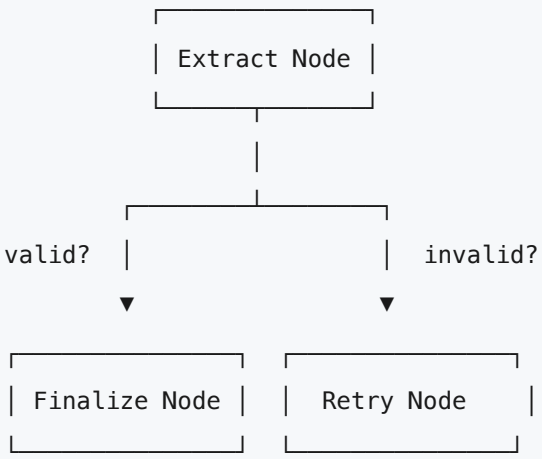
An **edge** defines which node runs after another. LangGraph supports two kinds:

A **normal edge** always moves from one node to a specific next node — no decision involved. Step A always leads to Step B.

A **conditional edge** examines the current state and decides which of several possible next nodes to run. This is where branching logic lives — the workflow equivalent of an if/else statement.

```
def route_decision(state):
    if state.get("needs_verification"):
        return "verify_node"
    else:
        return "finalize_node"

graph.add_conditional_edges(
    "extract_node",
    route_decision,
    {
        "verify_node": "verify_node",
        "finalize_node": "finalize_node"
    }
)
```

Think of a conditional edge like a fork in a road with a signpost. The traveler (the workflow) reads the sign (the routing function), checks the current conditions (the state), and takes the path that matches. Without conditional edges, every workflow would be a straight line — incapable of adapting to what actually happened in a previous step.



---

4. Retry Handling: Recovering Without Crashing

LLM calls fail. APIs time out. A model occasionally returns malformed output that breaks downstream parsing. A workflow that has no answer for failure simply crashes the moment something goes wrong — which is unacceptable once an agent is doing real, multi-step work.

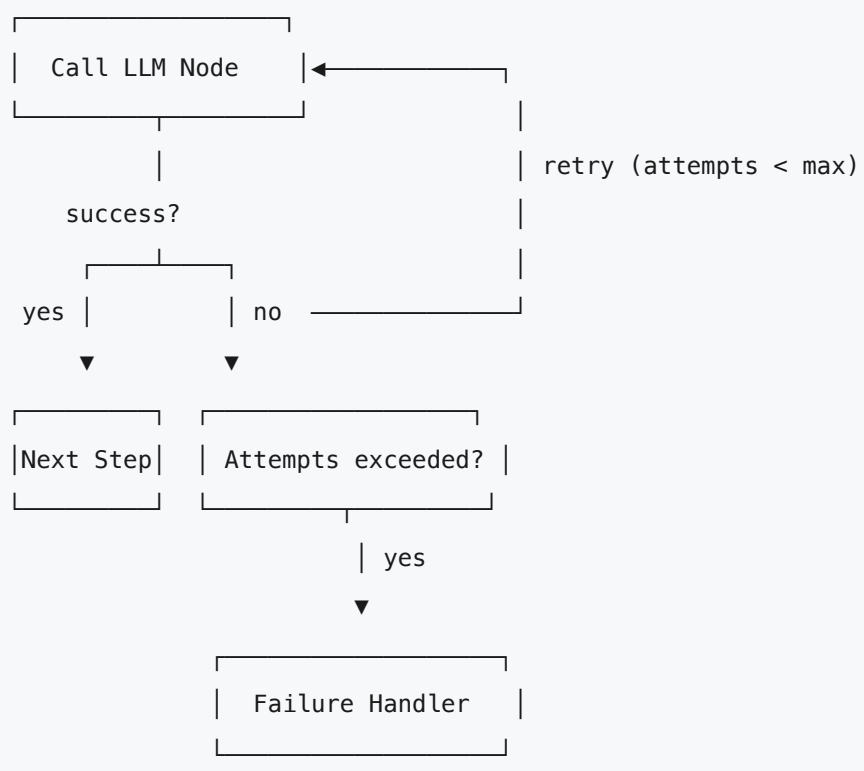
Retry handling is the mechanism by which a workflow detects a failed step and attempts it again — rather than propagating the failure forward or stopping entirely.

The simplest retry pattern wraps a node's logic in a loop with a maximum attempt count:

```
def call_with_retry(state, max_retries=3):
    attempts = state.get("attempts", 0)

    try:
        result = call_llm(state["input"])
        state["output"] = result
        state["success"] = True
        return state
    except Exception as e:
        attempts += 1
        state["attempts"] = attempts
        state["success"] = False
        if attempts >= max_retries:
            state["final_failure"] = True
        return state
```

The corresponding conditional edge decides what happens next based on whether the attempt succeeded:

```
def retry_router(state):
    if state.get("success"):
        return "next_step"
    elif state.get("final_failure"):
        return "failure_handler"
    else:
        return "call_with_retry"   # loop back and try again

graph.add_conditional_edges(
    "call_with_retry",
    retry_router,
    {
        "next_step": "next_step",
        "failure_handler": "failure_handler",
        "call_with_retry": "call_with_retry"
    }
)
```



A well-designed retry mechanism always has two things: a **maximum attempt limit** (so the workflow never loops forever) and a **clear failure path** (so when retries are exhausted, the workflow degrades gracefully — returning a fallback response or alerting a human — rather than crashing).

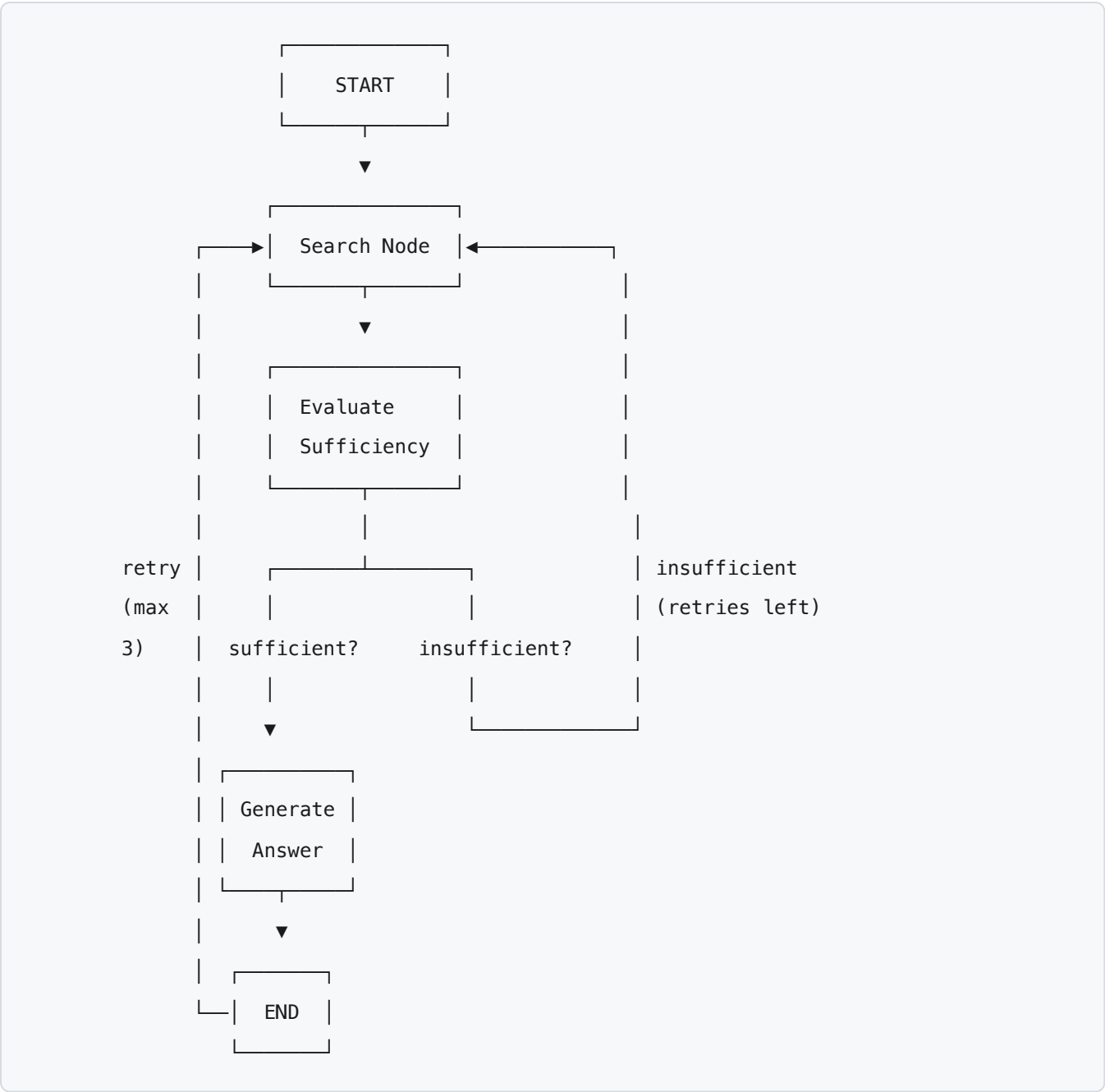
Tip: Always log the number of attempts and the reason for each failure inside the state. When a workflow does eventually fail after retries, that log is what lets you debug what actually went wrong, rather than just seeing a final dead end.

Common Mistake: Retrying on every type of failure indiscriminately. Some failures — like a malformed API key or an invalid input format — will never succeed no matter how many times you retry. Distinguish between transient failures (network timeout, rate limit) which are worth retrying, and permanent failures (bad input, missing config) which should fail fast instead.

---

5. Designing a Simple Agent Flow

Bringing nodes, conditional edges, and retry logic together produces a complete, working agent workflow. Consider a simple research agent: given a question, it searches for information, checks whether the answer is sufficient, and either finalizes the answer or searches again.


START
Search Node
Evaluate
Sufficiency
retry
(max
3)
sufficient?
insufficient?
insufficient
(retries left)
Generate
Answer
END

```
from langgraph.graph import StateGraph, END

class ResearchState(dict):
    question: str
    search_results: str
    attempts: int
    sufficient: bool
    answer: str

def search_node(state):
    state["search_results"] = run_search(state["question"])
    state["attempts"] = state.get("attempts", 0) + 1
    return state

def evaluate_node(state):
    state["sufficient"] = check_if_sufficient(state["search_results"])
    return state

def generate_answer_node(state):
    state["answer"] = call_llm(state["question"], state["search_results"])
    return state

def route_after_evaluate(state):
    if state["sufficient"]:
        return "generate_answer"
    elif state["attempts"] >= 3:
        return "generate_answer"   # give up and answer with what we have
    else:
        return "search"            # try searching again

graph = StateGraph(ResearchState)
graph.add_node("search", search_node)
graph.add_node("evaluate", evaluate_node)
graph.add_node("generate_answer", generate_answer_node)

graph.set_entry_point("search")
graph.add_edge("search", "evaluate")
graph.add_conditional_edges(
    "evaluate",
```

```
    route_after_evaluate,
    {"search": "search", "generate_answer": "generate_answer"}
)
graph.add_edge("generate_answer", END)

app = graph.compile()
result = app.invoke({"question": "What is LangGraph used for?", "attempts": 0})
```

Notice how this small example combines everything covered: nodes that each do one job, a conditional edge that branches based on state, and a retry-style loop with a maximum attempt cap to guarantee the workflow eventually terminates.

When designing your own agent flow, a useful checklist:

```
1. List every distinct step your task requires — each becomes a node.
2. For each step, ask: "what could go wrong here?" — that informs retry logic.
3. Identify points where the next step depends on the outcome of the
   current one — those become conditional edges.
4. Make sure every loop in your graph has a maximum iteration count.
5. Define what "done" looks like — every path should eventually reach END.
```

Key Takeaways

- **Orchestration** breaks a complex LLM task into smaller, well-defined steps instead of relying on a single large prompt. This makes workflows easier to debug, retry, and extend.
- **LangGraph** models a workflow as a graph of **nodes** (units of work) connected by **edges** (transitions), with a shared **state** object that flows through and accumulates data at every step.
- **Normal edges** always lead to the same next node. **Conditional edges** examine the current state and route to different nodes depending on what happened — this is how branching logic is built into a workflow.
- **Retry handling** detects failed steps and re-attempts them rather than letting the whole workflow crash. Every retry mechanism needs a maximum attempt limit and a defined failure path for when retries are exhausted.
- Not all failures should be retried. Transient failures (timeouts, rate limits) are worth retrying. Permanent failures (bad input, missing configuration) should fail fast instead of looping uselessly.

- A complete agent flow combines nodes for each distinct task, conditional edges for decision points, and retry loops with capped iterations — always designed so every possible path eventually reaches a terminal `END` state.
- When designing a new agent workflow, start by listing the discrete steps required, then identify where outcomes diverge (conditional edges) and where failures are likely (retry logic), before writing any code.

Mental Model

Think of a LangGraph workflow as an assembly line with built-in quality control. Each station on the line is a node, doing exactly one job. The conveyor belt between stations is the state, carrying the product forward with whatever has been added so far. At certain stations, an inspector checks the work and decides — based on what they see — whether the product moves to the next station or gets sent back for rework, up to a limited number of times before it's pulled aside entirely. Nothing on this line runs blind: every station knows its job, every decision point has a clear rule, and every product either reaches the end of the line successfully or is flagged for human attention. That is the difference between a single chef trying to do everything alone and a coordinated team that knows exactly when to redo a step instead of ruining the whole dish.